9. strcmp(): It compare two string using ascii values and returns the first ascii difference.

Strcmp(string1, string2);

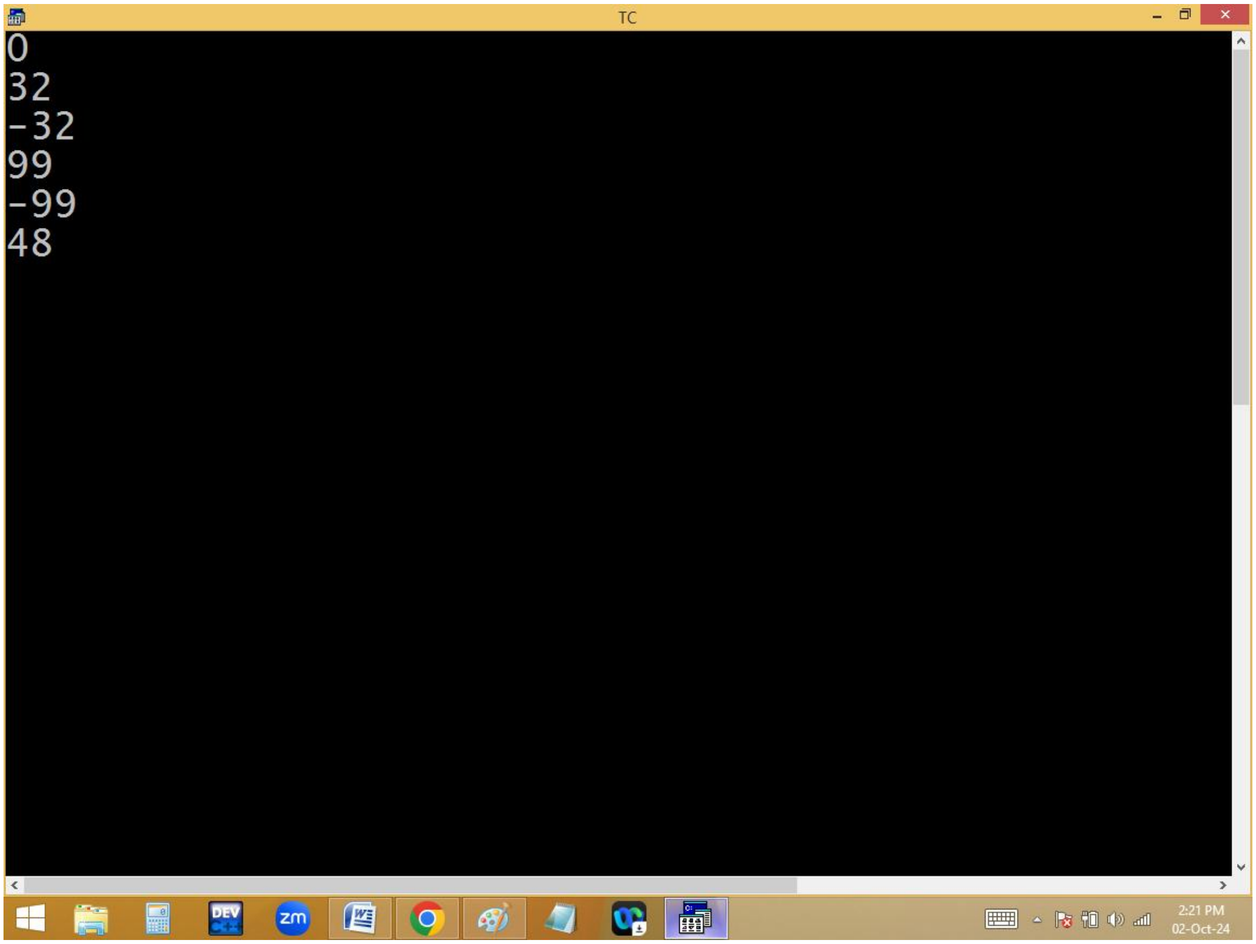
TC

File Edit Run Compile Project Options

Line 12 Col 32 Insert Indent Tab Fill Unir

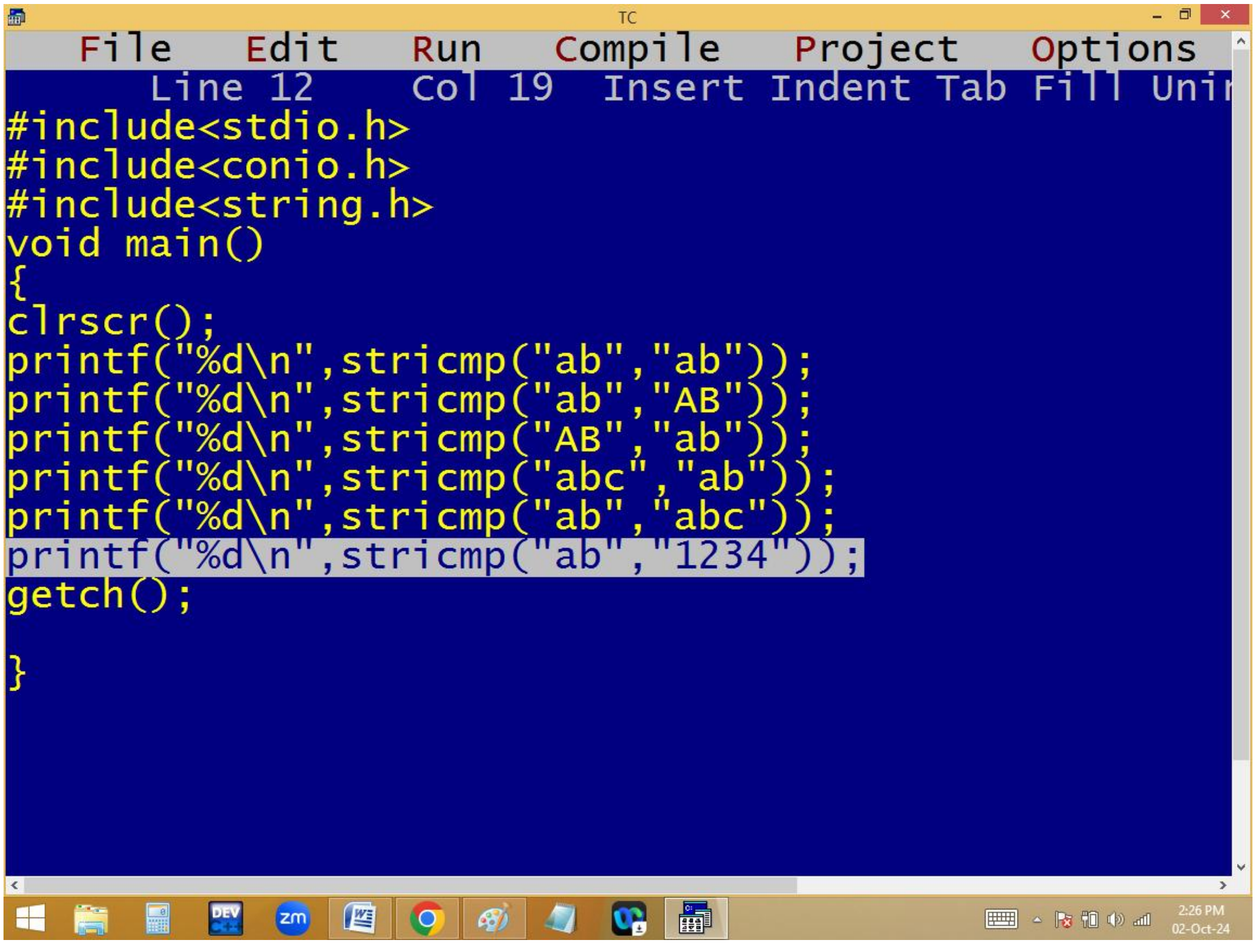
```c
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
clrscr();
printf("%d\n",strcmp("ab","ab"));
printf("%d\n",strcmp("ab","AB"));
printf("%d\n",strcmp("AB","ab"));
printf("%d\n",strcmp("abc","ab"));
printf("%d\n",strcmp("ab","abc"));
printf("%d\n",strcmp("ab","1234"));
getch();

}
```

2:21 PM
02-Oct-24

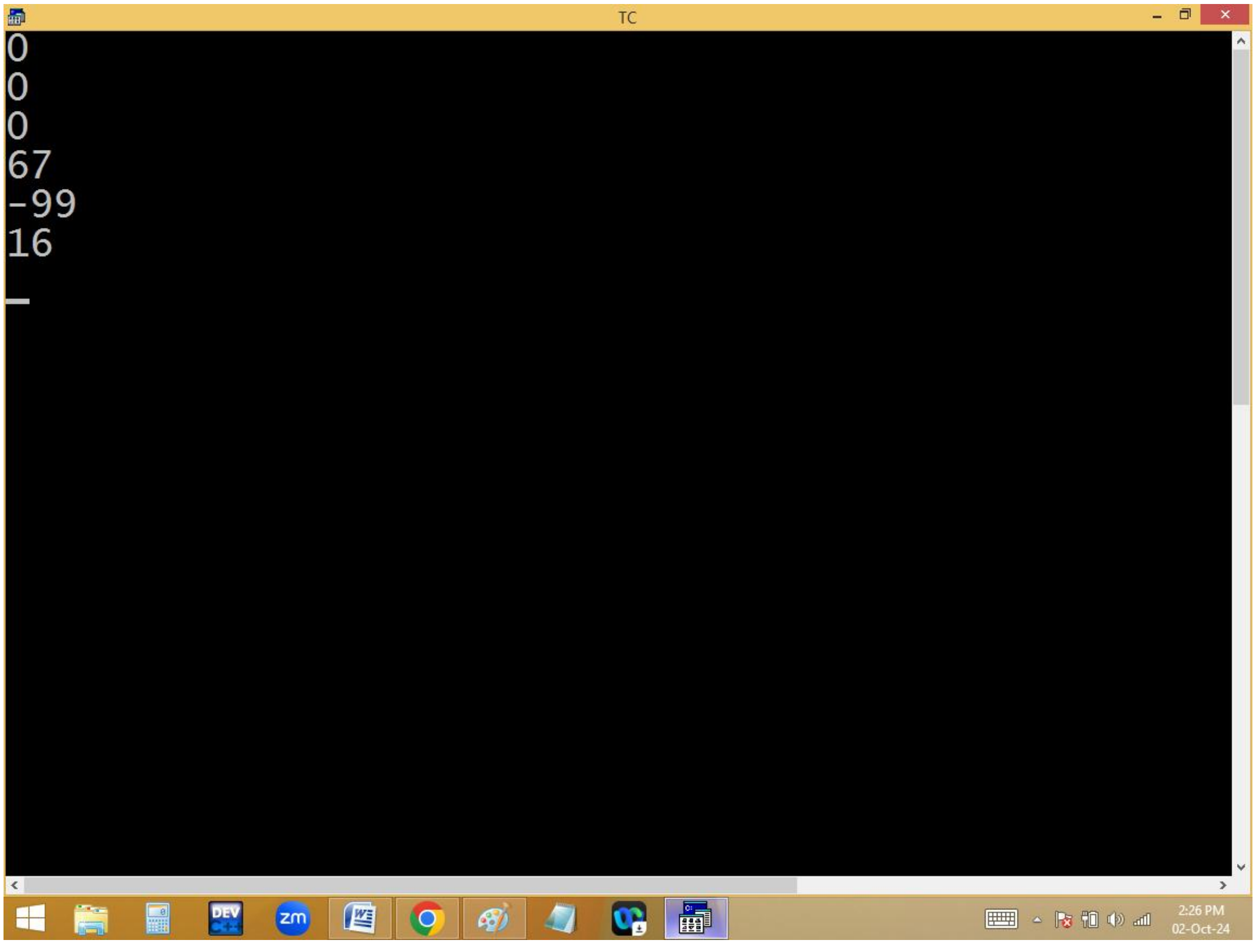


10. stricmp(): it compare two strings by **ignore case**. i.e. in stricmp lower and upper are same. If matching char not found or different data type found in 2nd string, the first string character taken in upper case.

stricmp(string1, string2);

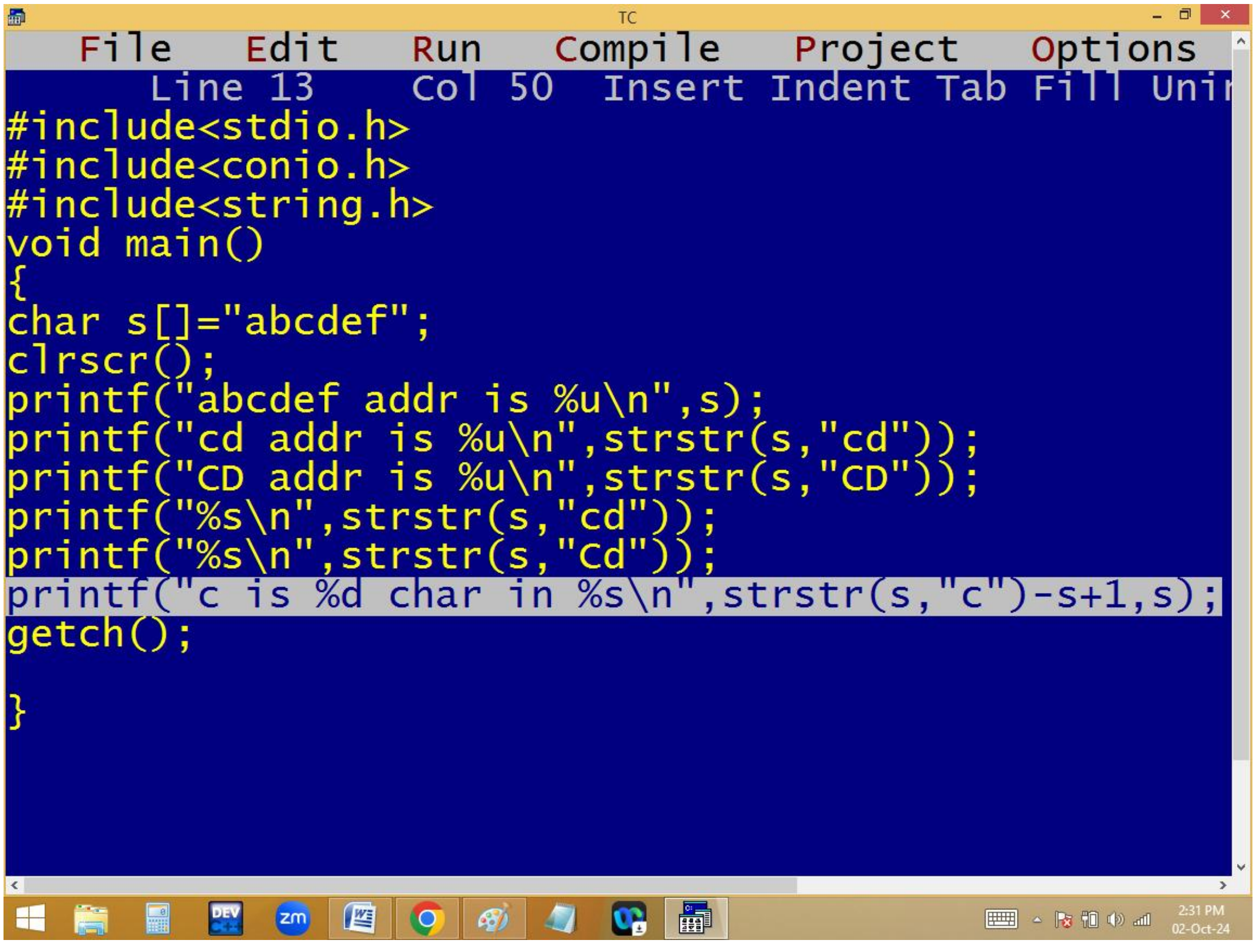



```c
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
clrscr();
printf("%d\n",stricmp("ab","ab"));
printf("%d\n",stricmp("ab","AB"));
printf("%d\n",stricmp("AB","ab"));
printf("%d\n",stricmp("abc","ab"));
printf("%d\n",stricmp("ab","abc"));
printf("%d\n",stricmp("ab","1234"));
getch();

}
```

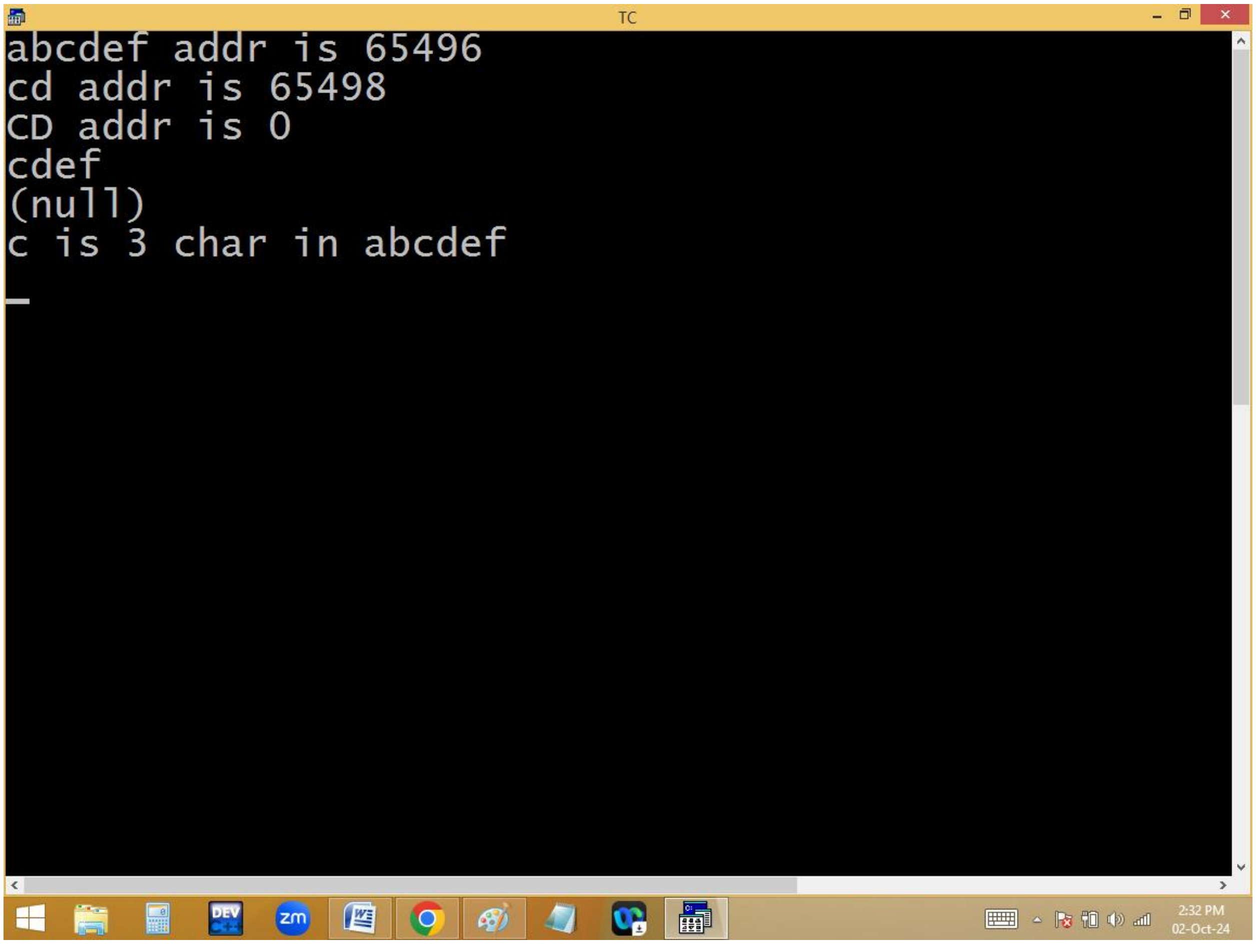


11. **Strstr()**: In returns the sub string address in main string. If sub string not found, it return 0.

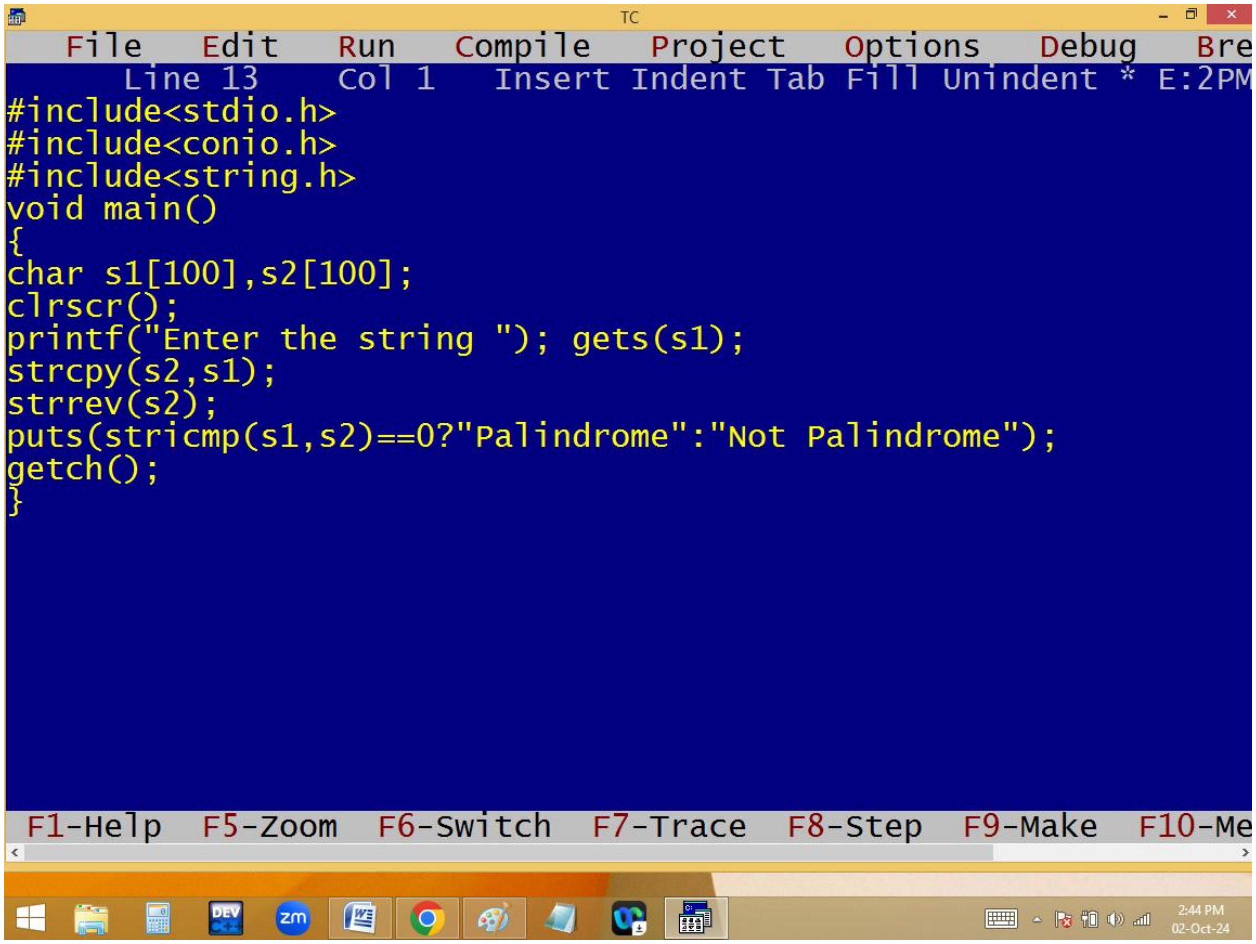
Strstr(main string, sub string);


TC
File Edit Run Compile Project Options
Line 13 Col 50 Insert Indent Tab Fill Unin
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
char s[]="abcdef";
clrscr();
printf("abcdef addr is %u\n",s);
printf("cd addr is %u\n",strstr(s,"cd"));
printf("CD addr is %u\n",strstr(s,"CD"));
printf("%s\n",strstr(s,"cd"));
printf("%s\n",strstr(s,"Cd"));
printf("c is %d char in %s\n",strstr(s,"c")-s+1,s);
getch();

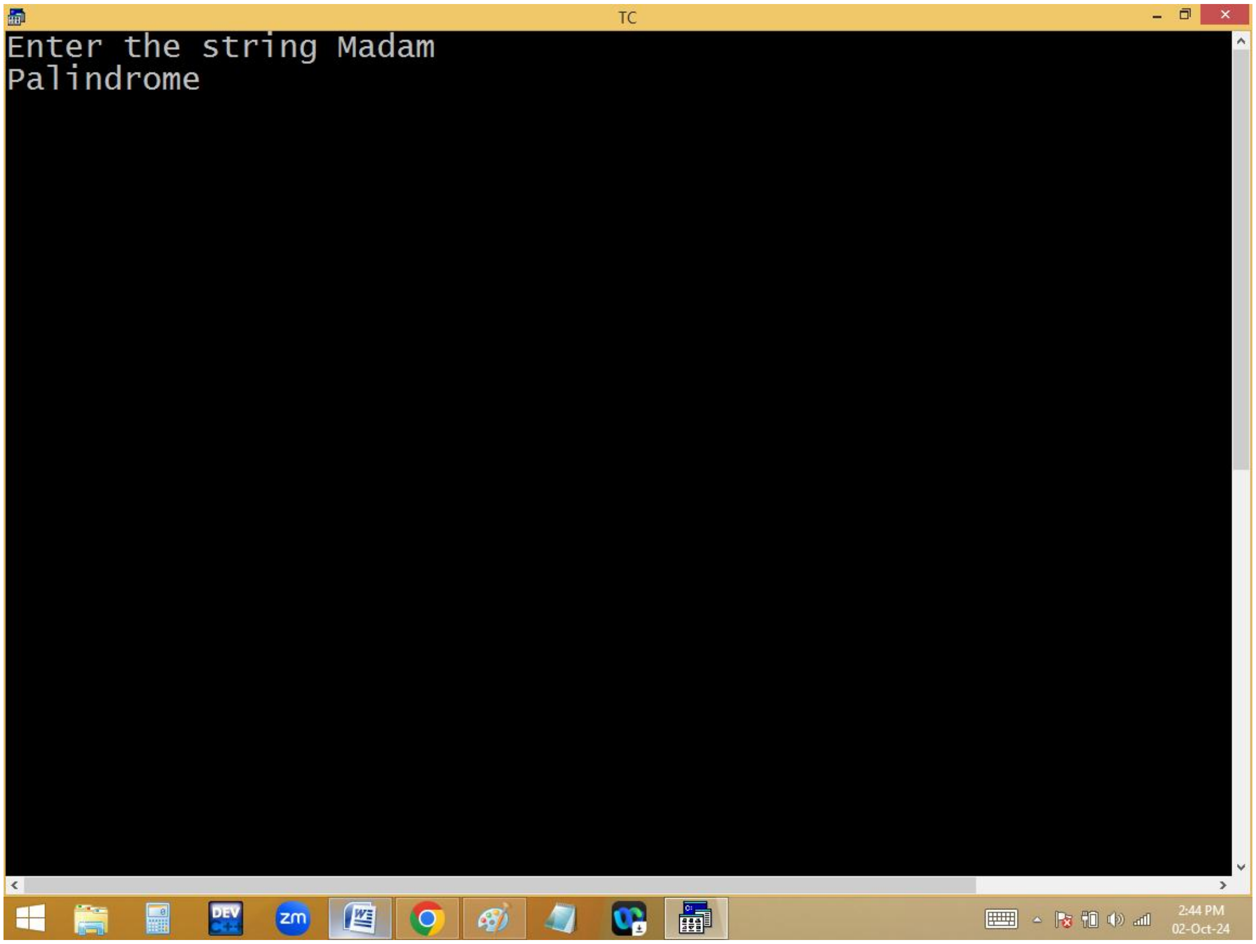
}
2:31 PM
02-Oct-24

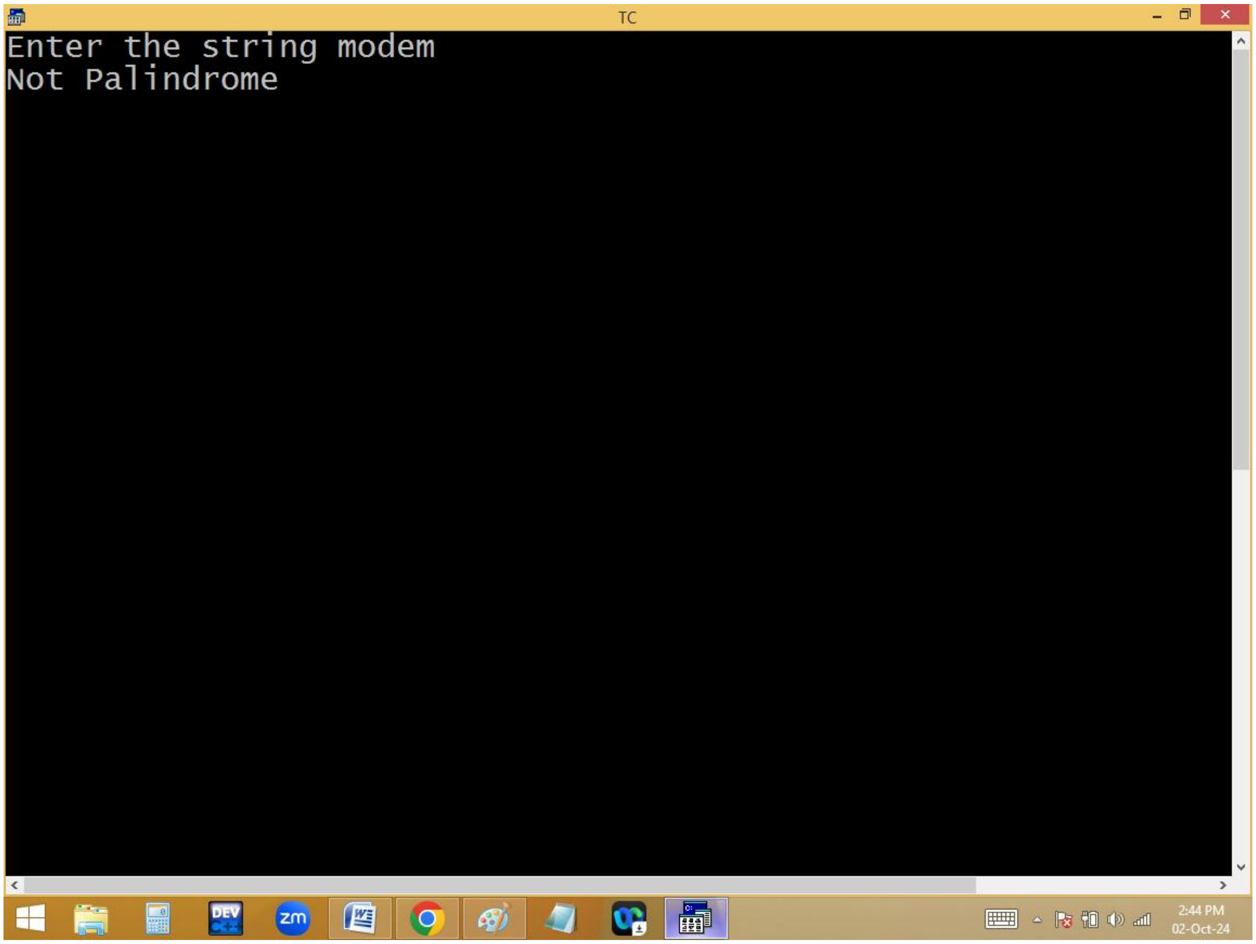


Eg: finding palindrome or not using library functions:

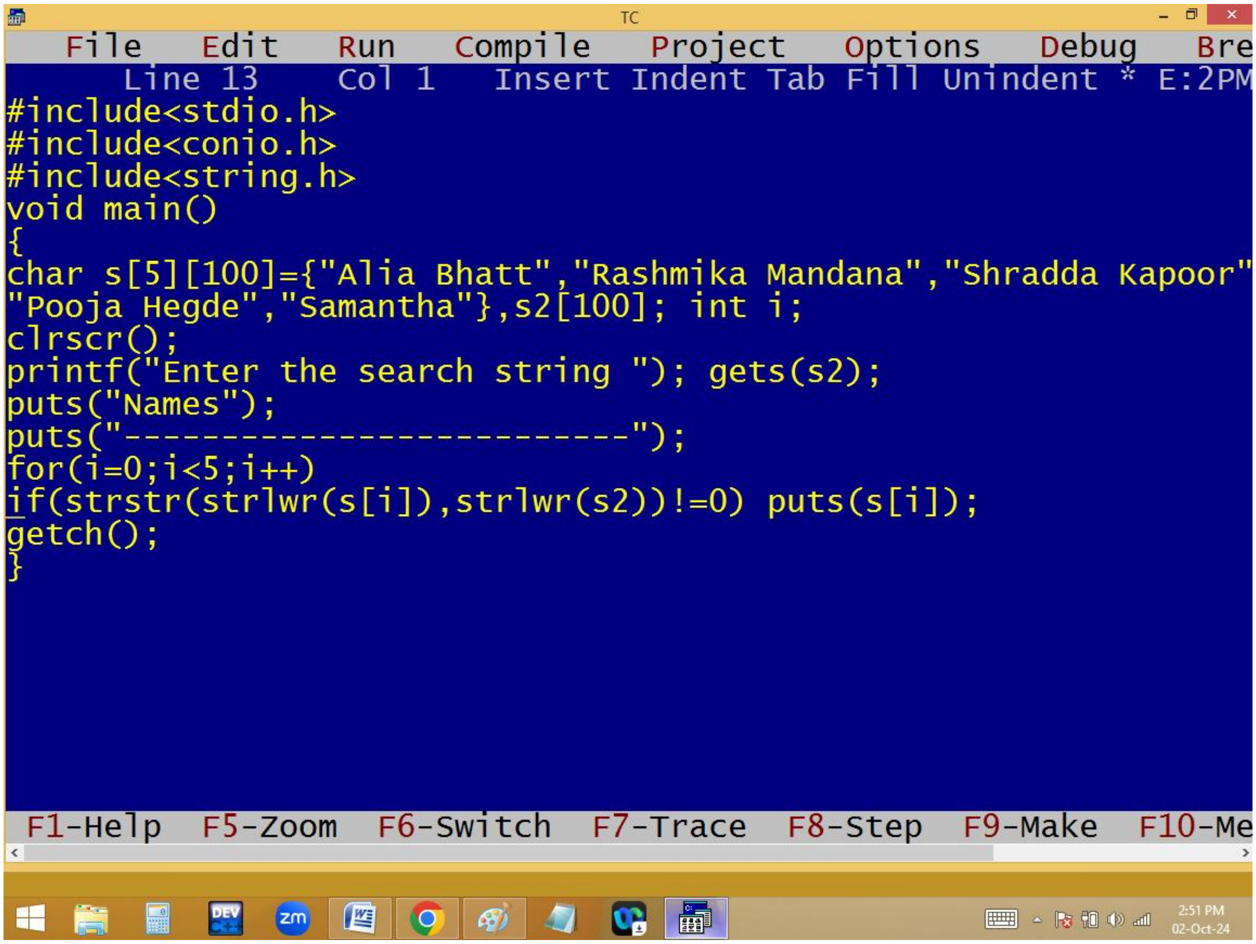


```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
char s1[100],s2[100];
clrscr();
printf("Enter the string "); gets(s1);
strcpy(s2,s1);
strrev(s2);
puts(stricmp(s1,s2)==0?"Palindrome":"Not Palindrome");
getch();
}
```

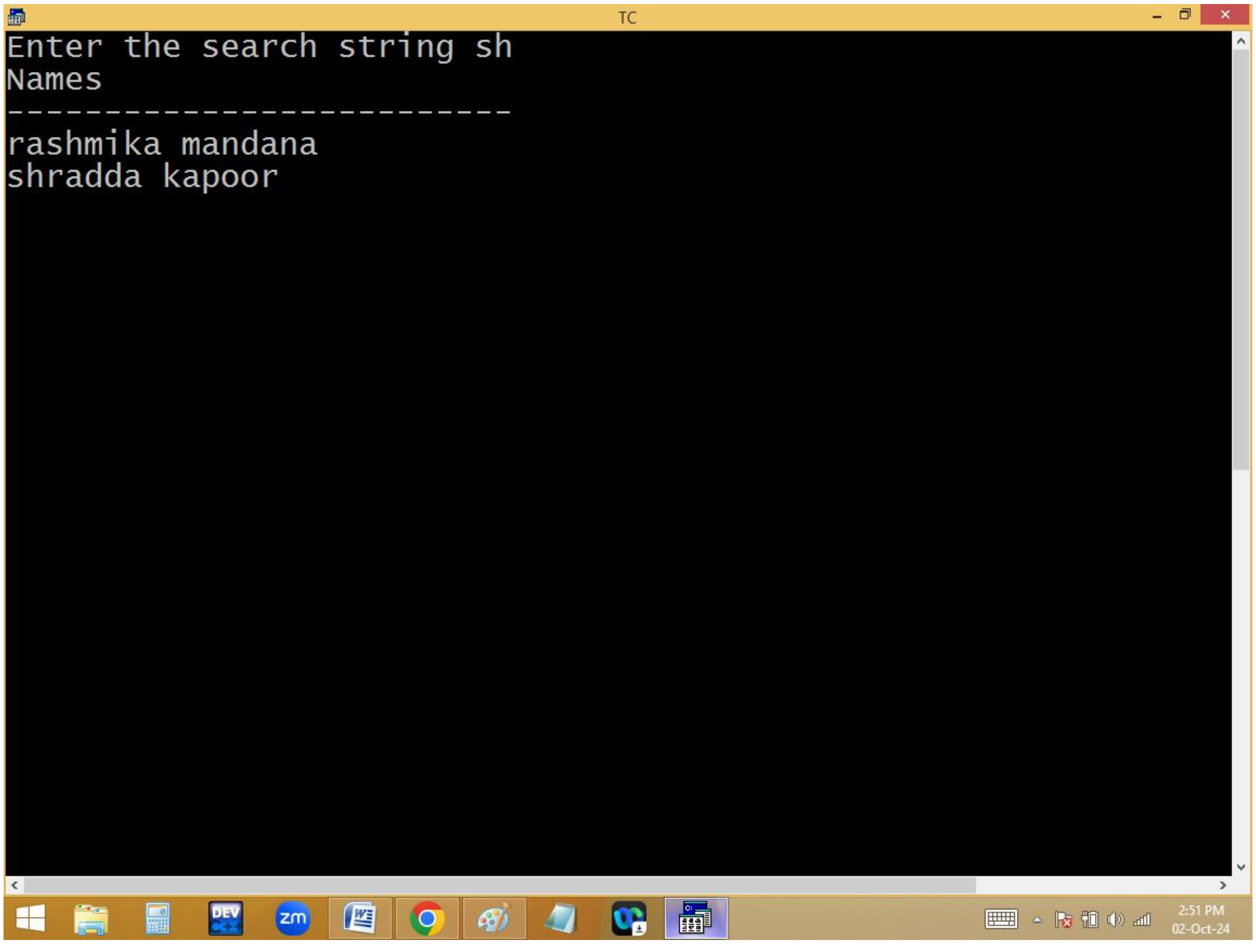

TC
Enter the string Madam
Palindrome
2:44 PM
02-Oct-24



2. **google searching working style:**




```c
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
char s[5][100]={"Alia Bhatt","Rashmika Mandana","Shradda Kapoor"
"Pooja Hegde","Samantha"},s2[100]; int i;
clrscr();
printf("Enter the search string "); gets(s2);
puts("Names");
puts("-------------------------");
for(i=0;i<5;i++)
if(strstr(strlwr(s[i]),strlwr(s2))!=0) puts(s[i]);
getch();
}
```



Sorting of strings:

s[0]	~~janu~~ ~~abhi~~ aayesha
s[1]	~~abhi~~ ~~janu~~ ~~chintu~~ abhi
s[2]	~~chintu~~ ~~janu~~ chintu
s[3]	~~aayesha~~ ~~abhi~~ ~~chintu~~ janu

TC

File Edit Run Compile Project Options Debug Bre

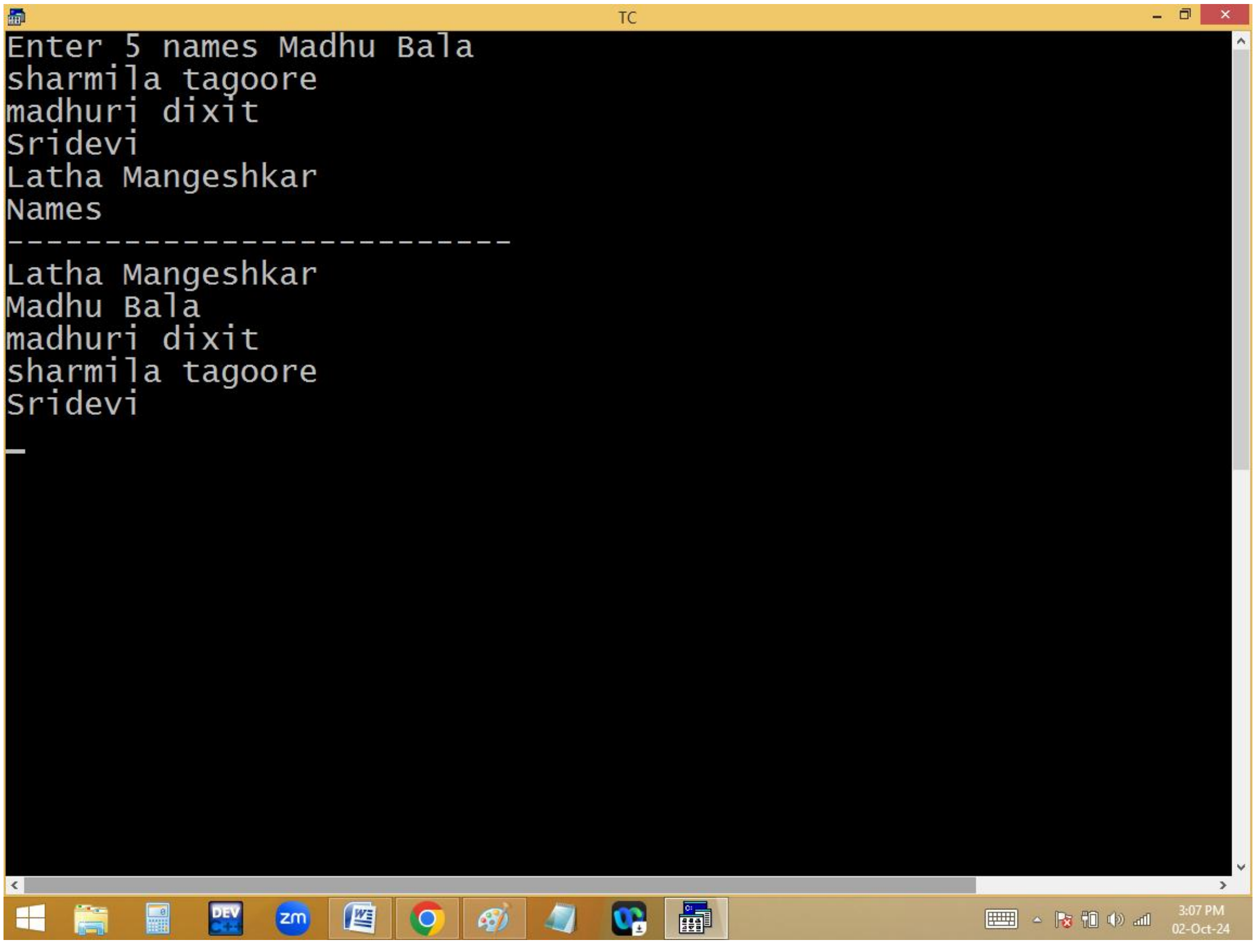
Line 1 Col 8 Insert Indent Tab Fill Unindent * E:2PM

```c
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
char s[5][100],t[100]; int i,j;
clrscr();
printf("Enter 5 names ");
for(i=0;i<5;i++)gets(s[i]);
for(i=0;i<=3;i++)
{
for(j=i+1;j<=4;j++)
{
if(stricmp(s[i],s[j])>0)
{strcpy(t,s[i]);strcpy(s[i],s[j]);strcpy(s[j],t);}
}
}
puts("Names");
puts("--------------------------");
for(i=0;i<5;i++)puts(s[i]);
getch();
}
```

F1-Help F5-Zoom F6-Switch F7-Trace F8-Step F9-Make F10-Me

1894 × 763px 100%

3:07 PM
02-Oct-24



USER DEFINED FUNCTIONS

What is a function?

1. It is a small program used to do a particular task.
2. It is a small program with in another program.
3. It is a sub program or sub routine.
4. It is a procedure.
5. It is a self-contained block.
6. It is a reusable code component.

7. It is a module.

Advantages:

1. **Modularity**: Dividing program instructions into small pieces.
2. **Simplicity**: It is easy to understand the program instructions.
3. **Reusability**: Just write once, use several times.
4. **Efficiency**: performance is improved.
5. It is easy to identify the errors.

   Entire 'C' program is collection of functions. Hence 'C' is a function oriented structured programming language.

We are having 2 types of functions.

1. Predefined / library / built-in functions

These are the functions provided with software and ready to use.

Eg: printf(), scanf(),…

2. **User defined functions**: They are created by the user.

Eg: sum(), prime(), factorial(),……

Every user defined function is divided into 3 parts.

1. Function declaration/ proto typing
2. Function calling
3. Function definition

Function declaration/proto typing:

Generally function declaration is conducted before or within the main(), before function calling.

It tells the compiler that we are going to use this kind of function in our program in future.

Syntax:

[return_datatype] function_name ([arguments / parameters]);

Eg:

void sum();

Function calling:

It should be conducted within the main() only.

When a function is invoked (called), the compiler will search for the matching function definition and it is available then the program execution is jumped from function calling area to the function definition area.

Linking a function call to the function definition is called **binding.** C language supports only static or compile time binding. But C++ supports static and dynamic binding[run time binding].

Without function call, a function never participates in function execution. Hence it is mandatory to call a function in a program.

<u>Syntax:</u>

function_name([arguments]);

Eg: sum();

<u>Function definition:</u>

It contains the function header and body. The function header should be matched with function declaration.

It is conducted outside the main(). If the definition is conducted before the main(), there is no need of function declaration.

Function header consists of function name and the parameters.

Function body consists of statements related to the task of function.

When a function is invoked, the program execution is shifted from function calling area (main()) to function definition area. After executing the function body, the execution is jumped to the next statement after the function call in main().

Function declaration and function definition should be identical.

Syntax:

Function Header

[return data type]
Function_name([parameters/arguments])

{

  Statements;

 [return value;]

}

Function Body

Here return data type indicates the type of value that the function is returning to main() / called function. It is depended on the return value.

If the sub program is having return statement then it is called function and without return value it is called procedure.

The **default return value of a function is integer**. Use the keyword **void** [nothing] when the function is not having any return value. Otherwise integer will become the return value.

A function may have any number of return statements. But only one return statement is executed at a time. The statements after the return statement are not executed and gives compile time warning.

Return data type should be matched with return value data type.

Function name is used to identify a function and it should not be predefined.

Arguments / parameters used to carry the values from function calling area to function definition area. They are optional and of two types.

1. **Actual** arguments/parameters.
2. **Formal** arguments/parameters.

The arguments we are sending in function calling at main() are called actual parameters and the parameters we are using in function definition to receive the actual argument values is called formal parameters.

Actual and formal parameter names may be same or different.

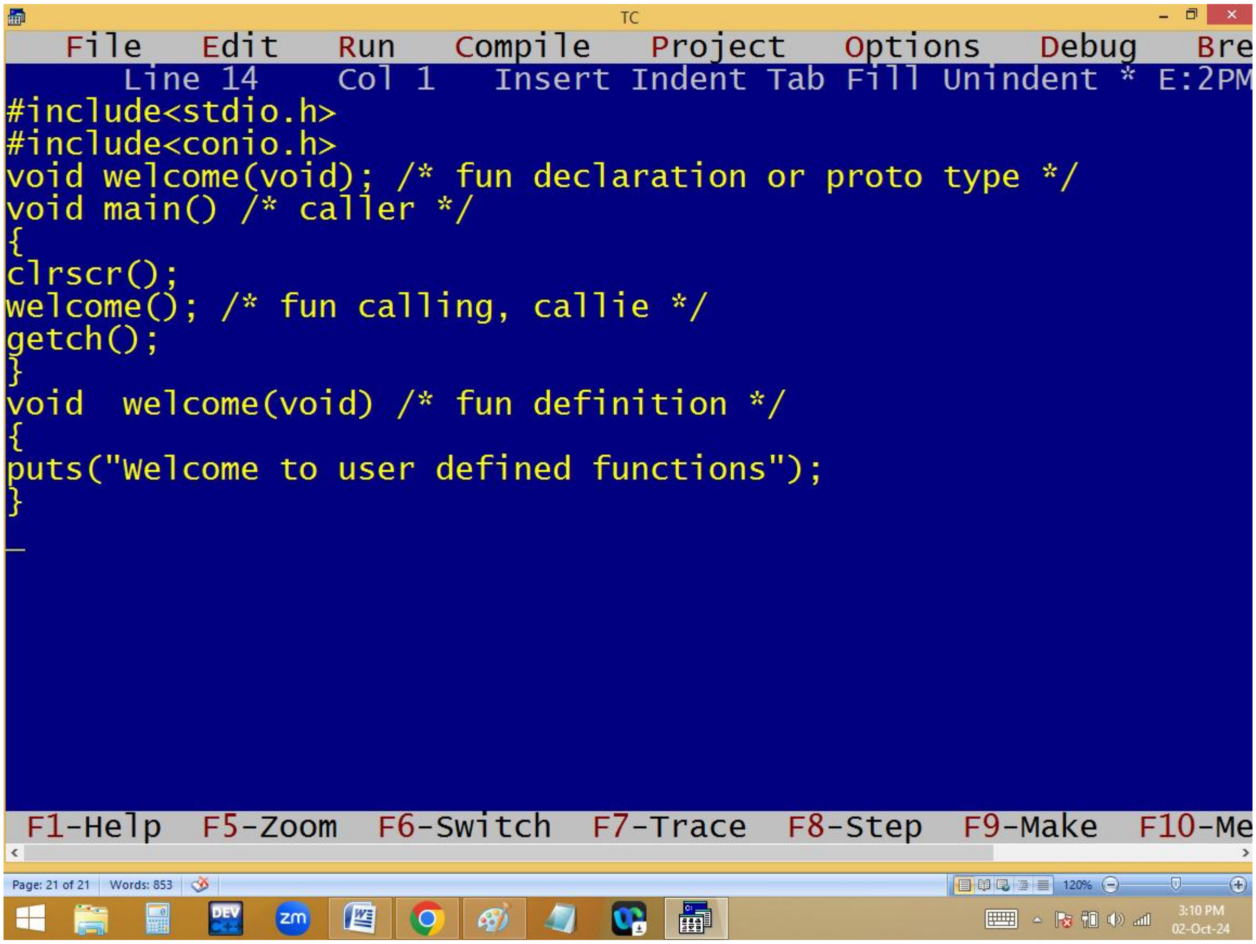
Difference between arguments and parameters is "arguments are nothing but different vehicles which are parked in same parking area and the parking area is the parameters. i.e. arguments are always changed but parameters are fixed.

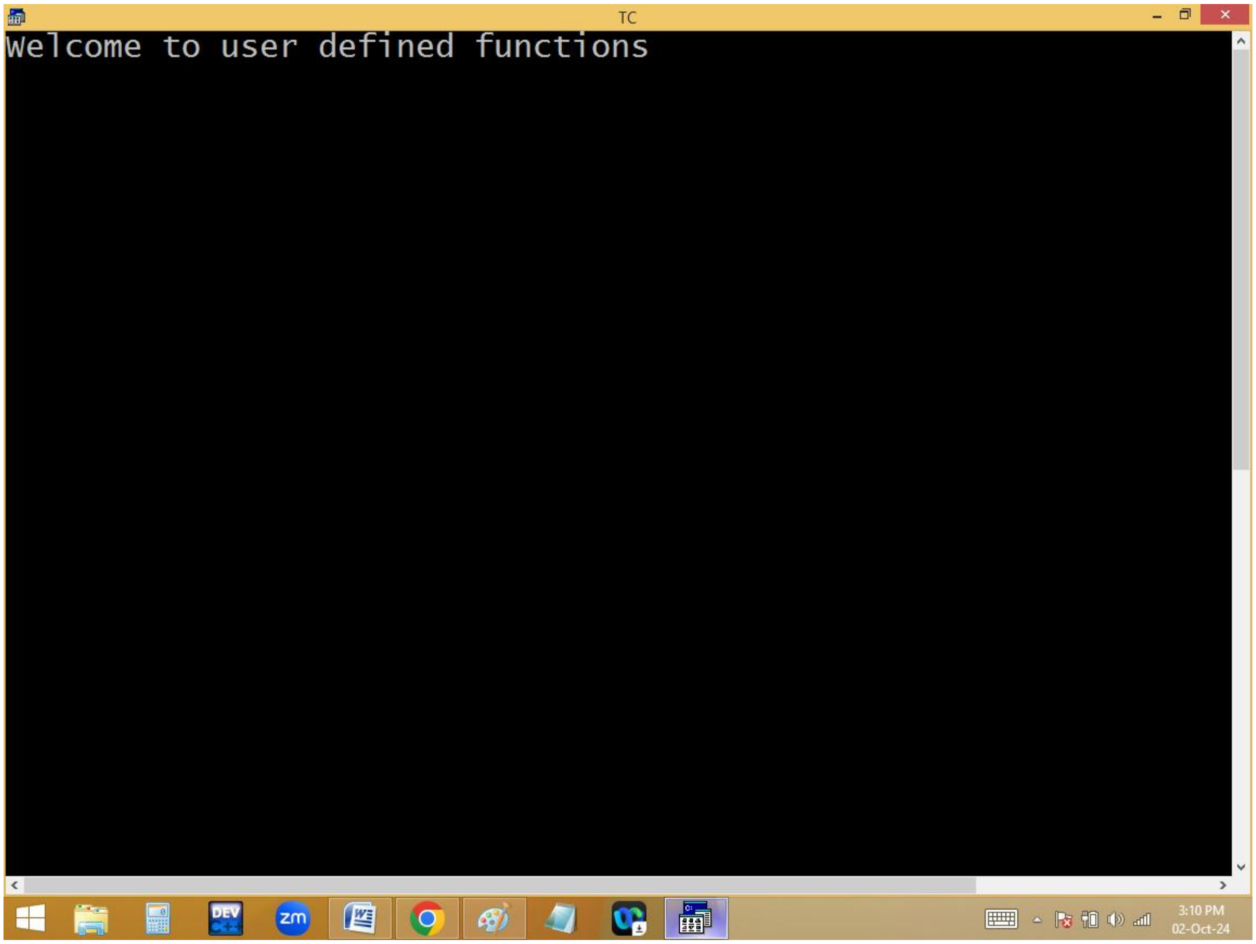
The calling function arguments should be matched with definition parameters list in quantity, data type and order.

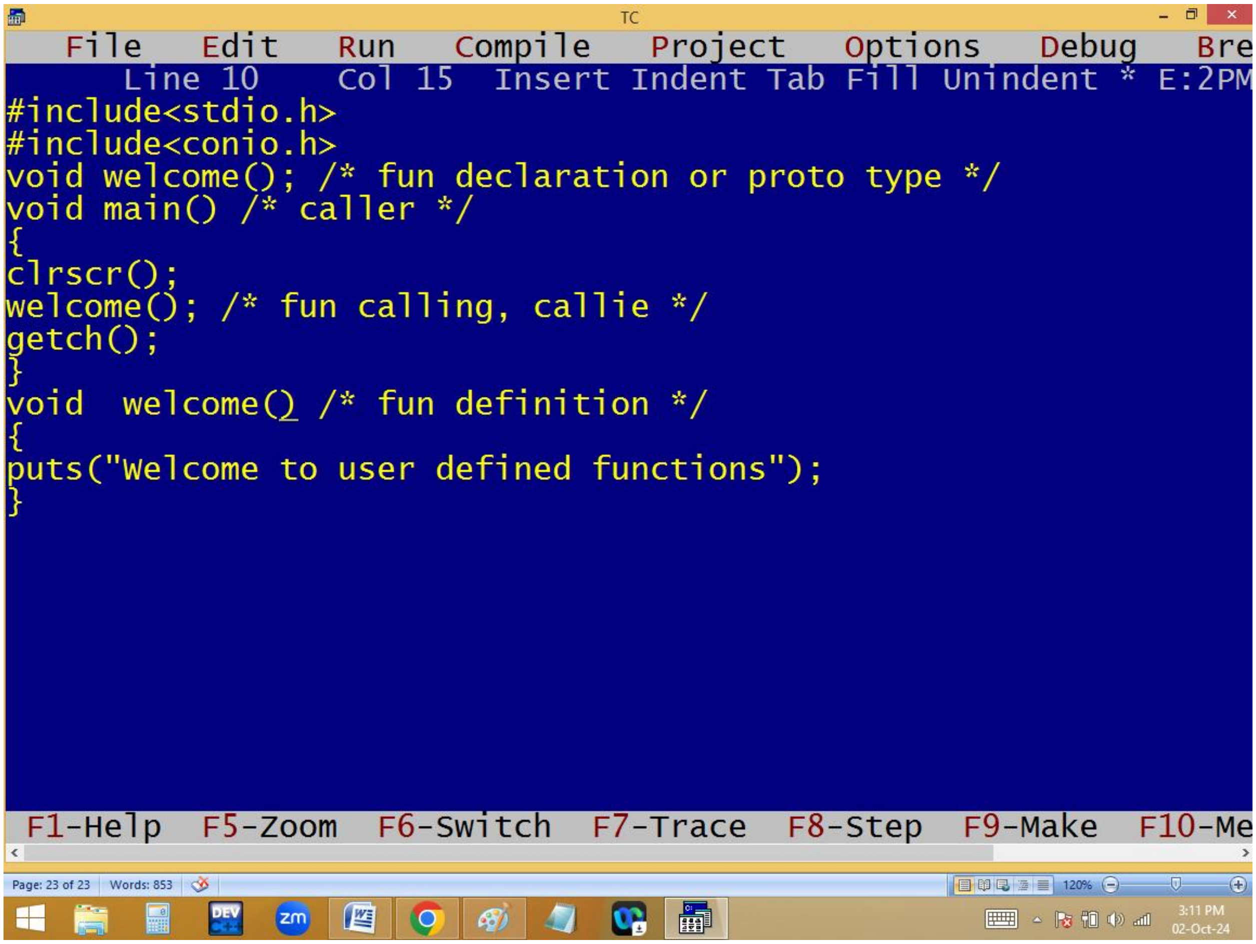
Based on arguments and return values, functions are divided into 4 types.

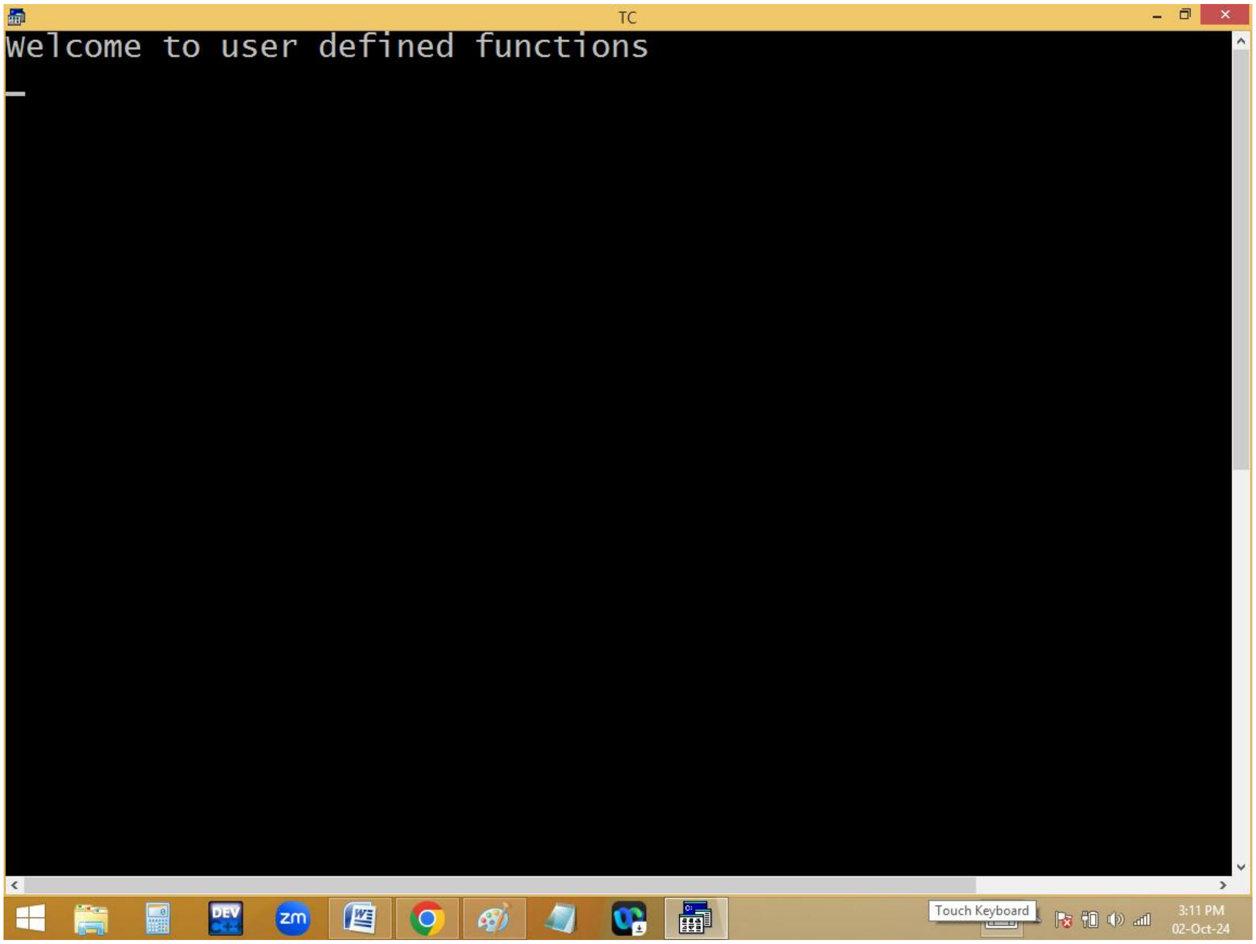
1. Function without arguments, without return values.
2. Function with arguments and with return values.
3. Function with arguments and without return values.
4. Function without arguments and with return values.

Eg: **function without arguments, without return values**


TC
File Edit Run Compile Project Options Debug Bre
Line 14 Col 1 Insert Indent Tab Fill Unindent * E:2PM
#include<stdio.h>
#include<conio.h>
void welcome(void); /* fun declaration or proto type */
void main() /* caller */
{
clrscr();
welcome(); /* fun calling, callie */
getch();
}
void welcome(void) /* fun definition */
{
puts("Welcome to user defined functions");
}
F1-Help F5-Zoom F6-Switch F7-Trace F8-Step F9-Make F10-Me
Page: 21 of 21 Words: 853
120%
3:10 PM
02-Oct-24


TC
Welcome to user defined functions
3:10 PM
02-Oct-24


TC
File Edit Run Compile Project Options Debug Bre
Line 10 Col 15 Insert Indent Tab Fill Unindent * E:2PM
#include<stdio.h>
#include<conio.h>
void welcome(); /* fun declaration or proto type */
void main() /* caller */
{
clrscr();
welcome(); /* fun calling, callie */
getch();
}
void welcome() /* fun definition */
{
puts("Welcome to user defined functions");
}
F1-Help F5-Zoom F6-Switch F7-Trace F8-Step F9-Make F10-Me
Page: 23 of 23 Words: 853
120%
3:11 PM
02-Oct-24


TC
Welcome to user defined functions
Touch Keyboard
3:11 PM
02-Oct-24

Turbo C++ IDE

≡ File Edit Search Run Compile Debug Project Options

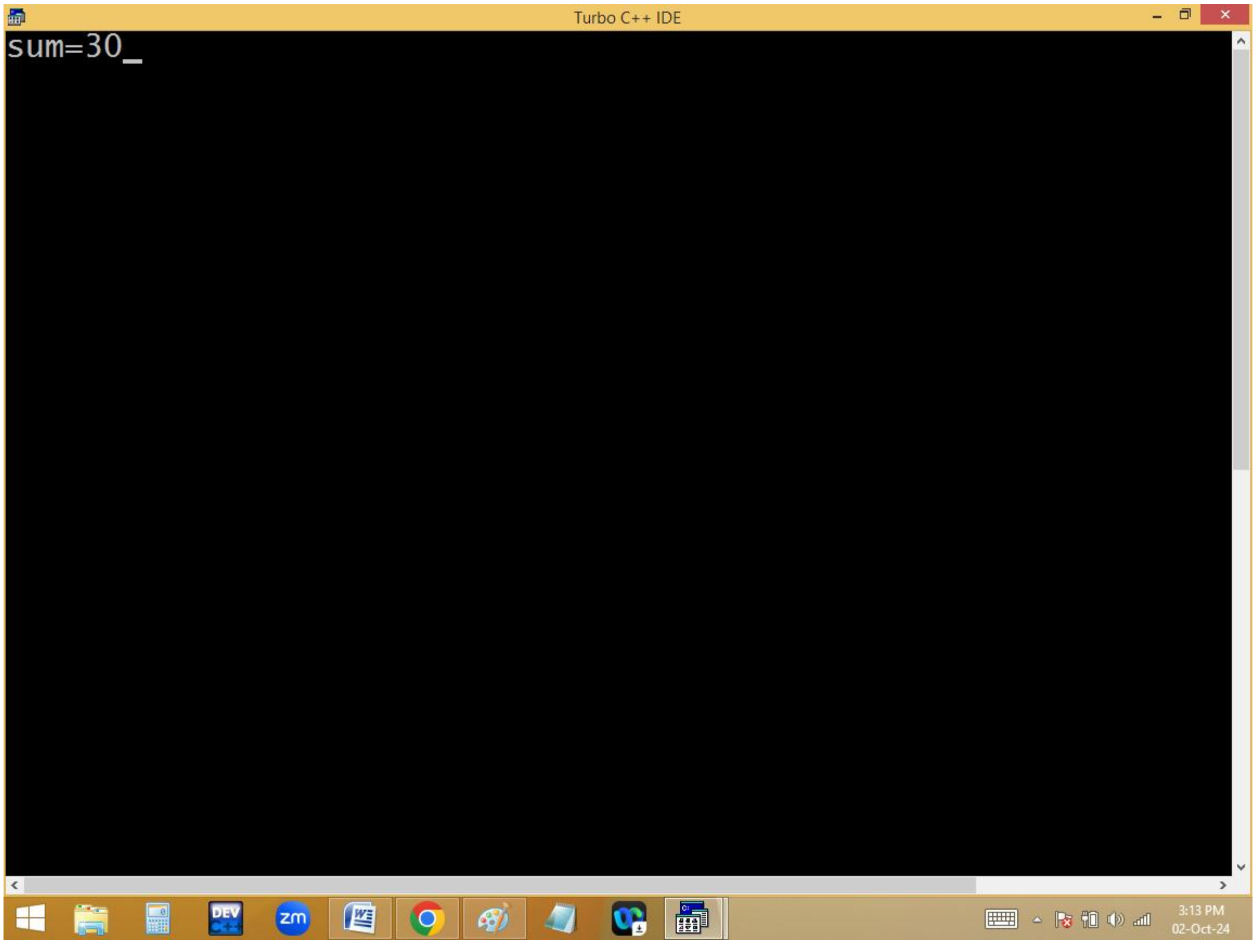
F1.CPP

```
#include<stdio.h>
#include<conio.h>
void  sum(); // fun dec  without arguments

void  main() // caller
{
clrscr();
sum(); // calling, callie
getch();
}

void  sum() // fun definition
{
int a=10, b=20;
printf("sum=%d",a+b);
}
```

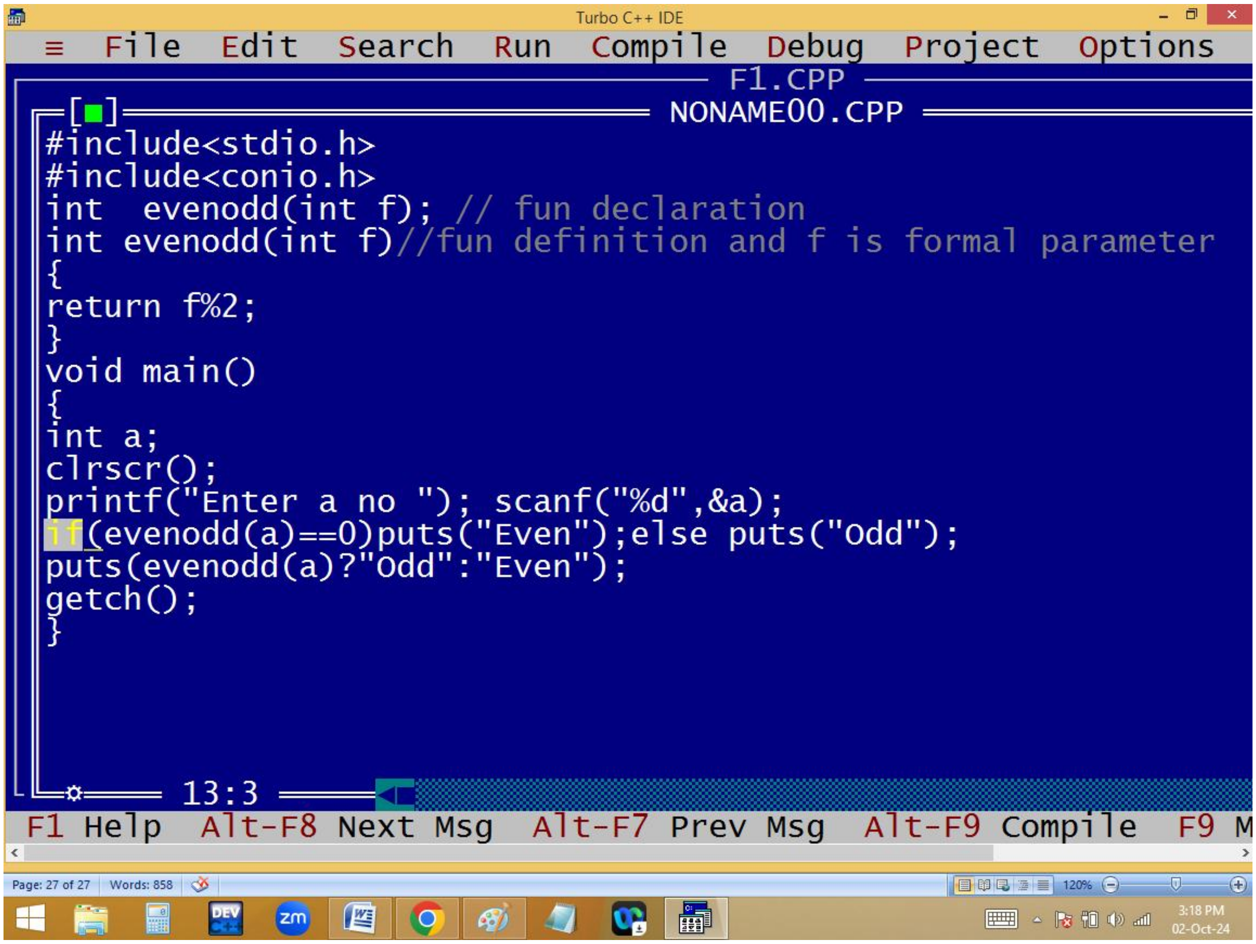
17:1

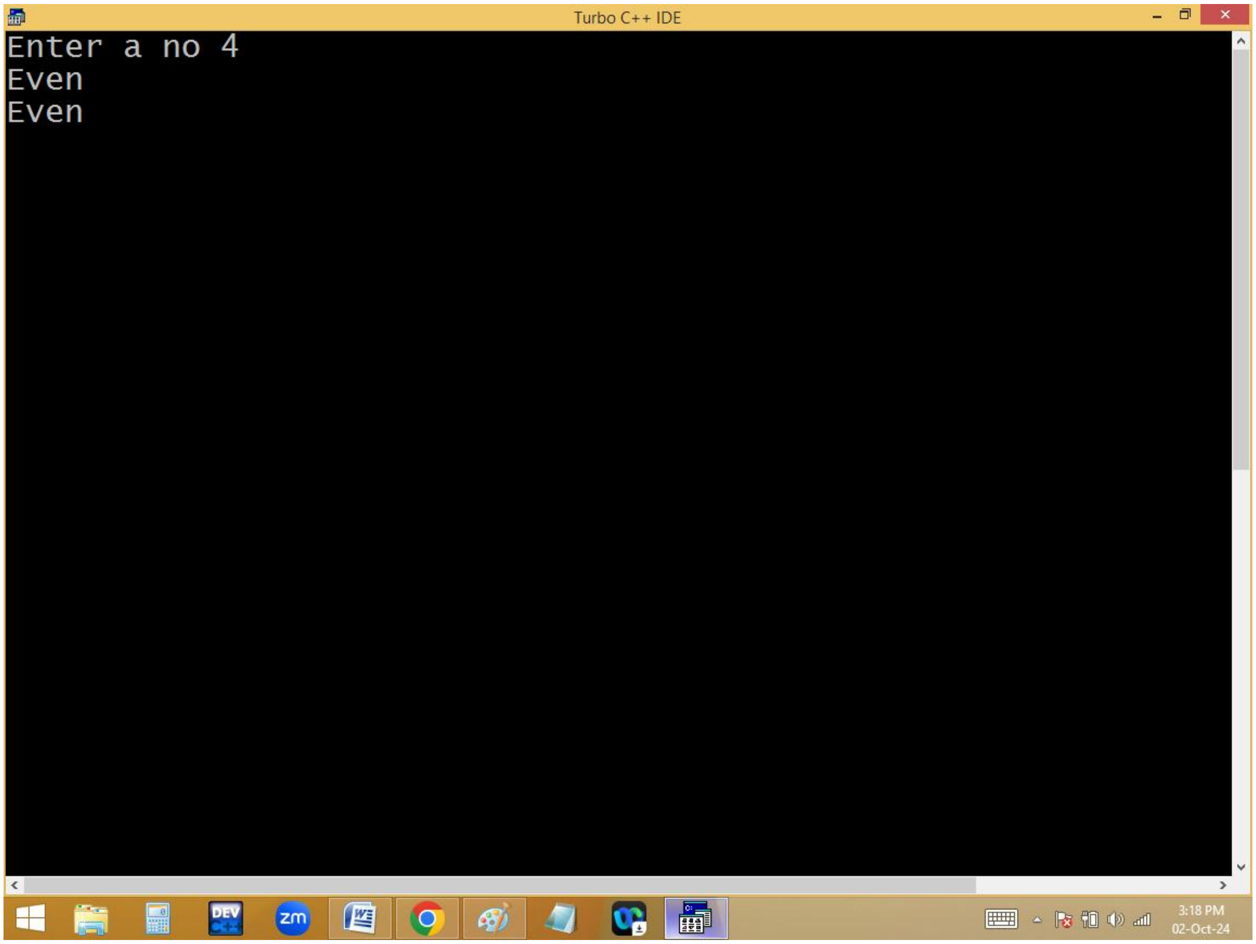
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M

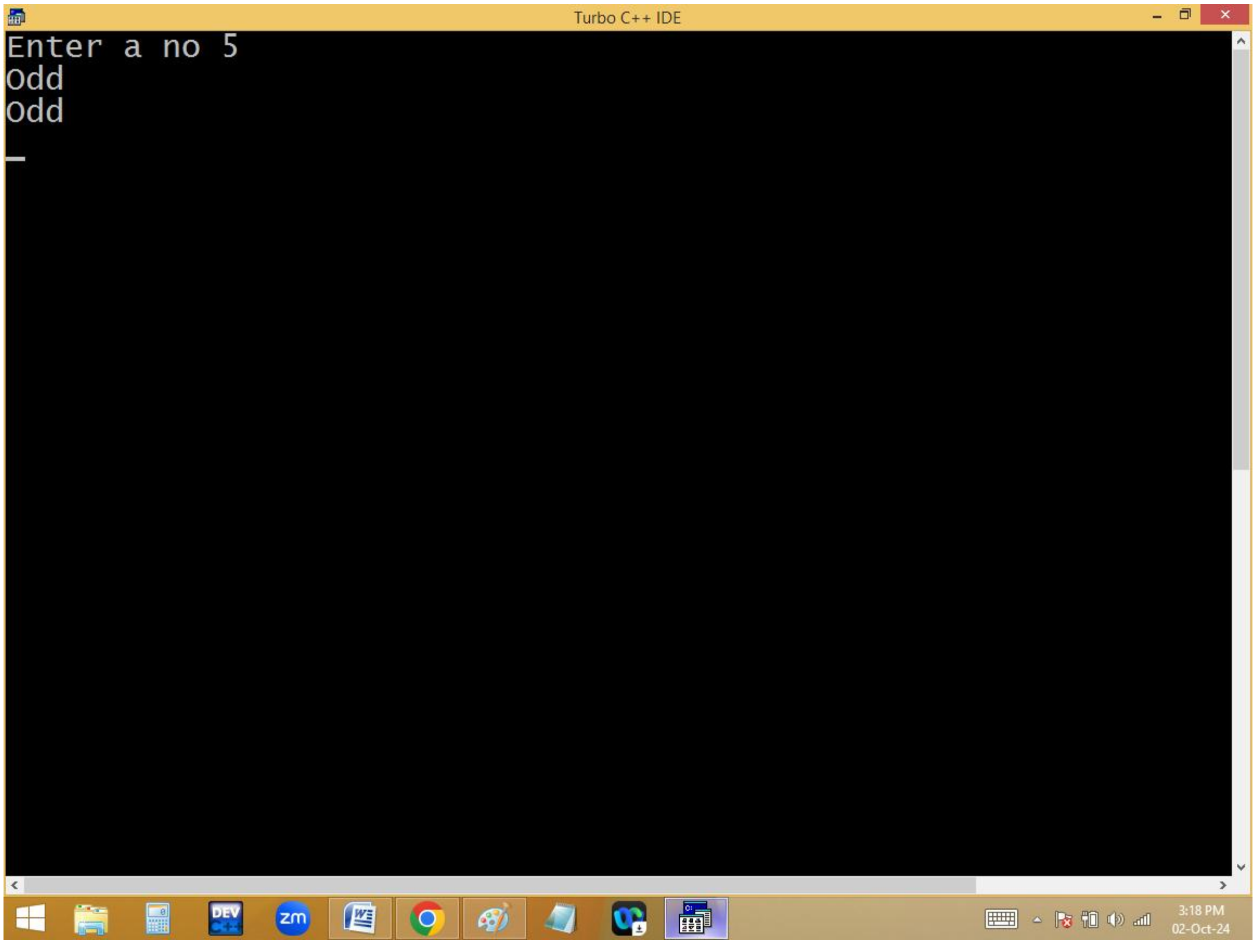
3:13 PM
02-Oct-24

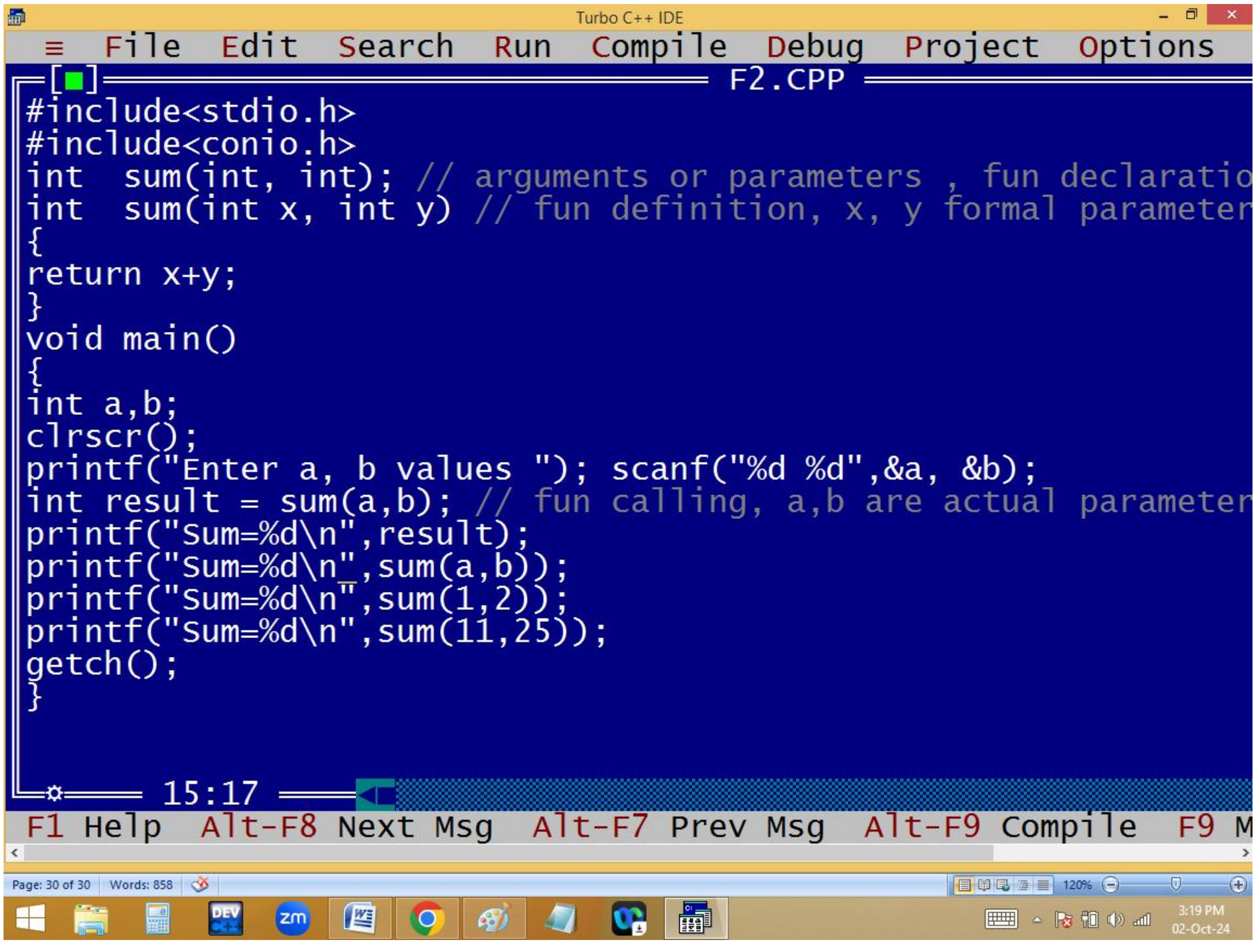


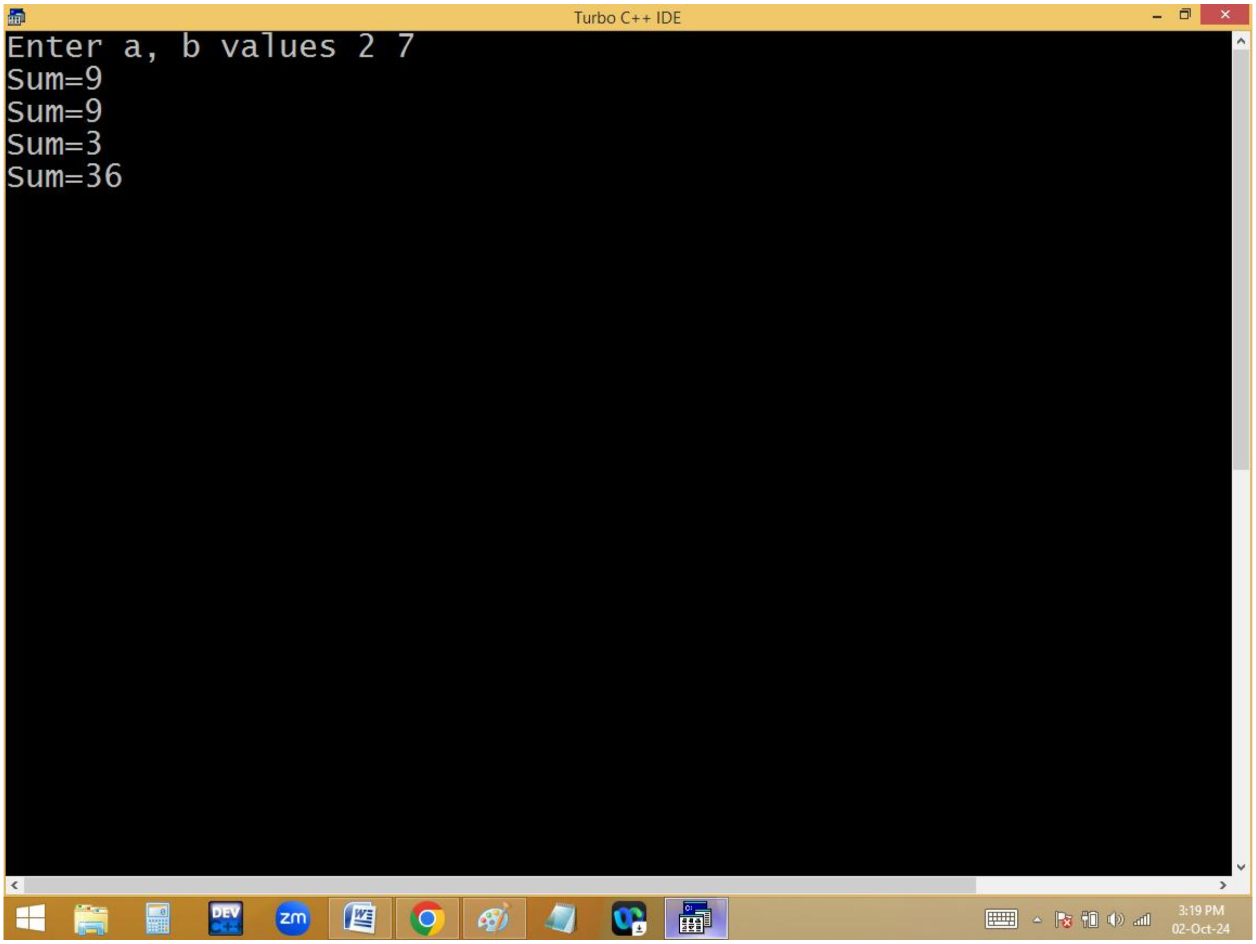
With arguments, with return value:


Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
F1.CPP
NONAME00.CPP
#include<stdio.h>
#include<conio.h>
int evenodd(int f); // fun declaration
int evenodd(int f)//fun definition and f is formal parameter
{
return f%2;
}
void main()
{
int a;
clrscr();
printf("Enter a no "); scanf("%d",&a);
if(evenodd(a)==0)puts("Even");else puts("Odd");
puts(evenodd(a)?"Odd":"Even");
getch();
}
13:3
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 27 of 27 Words: 858
120%
3:18 PM
02-Oct-24

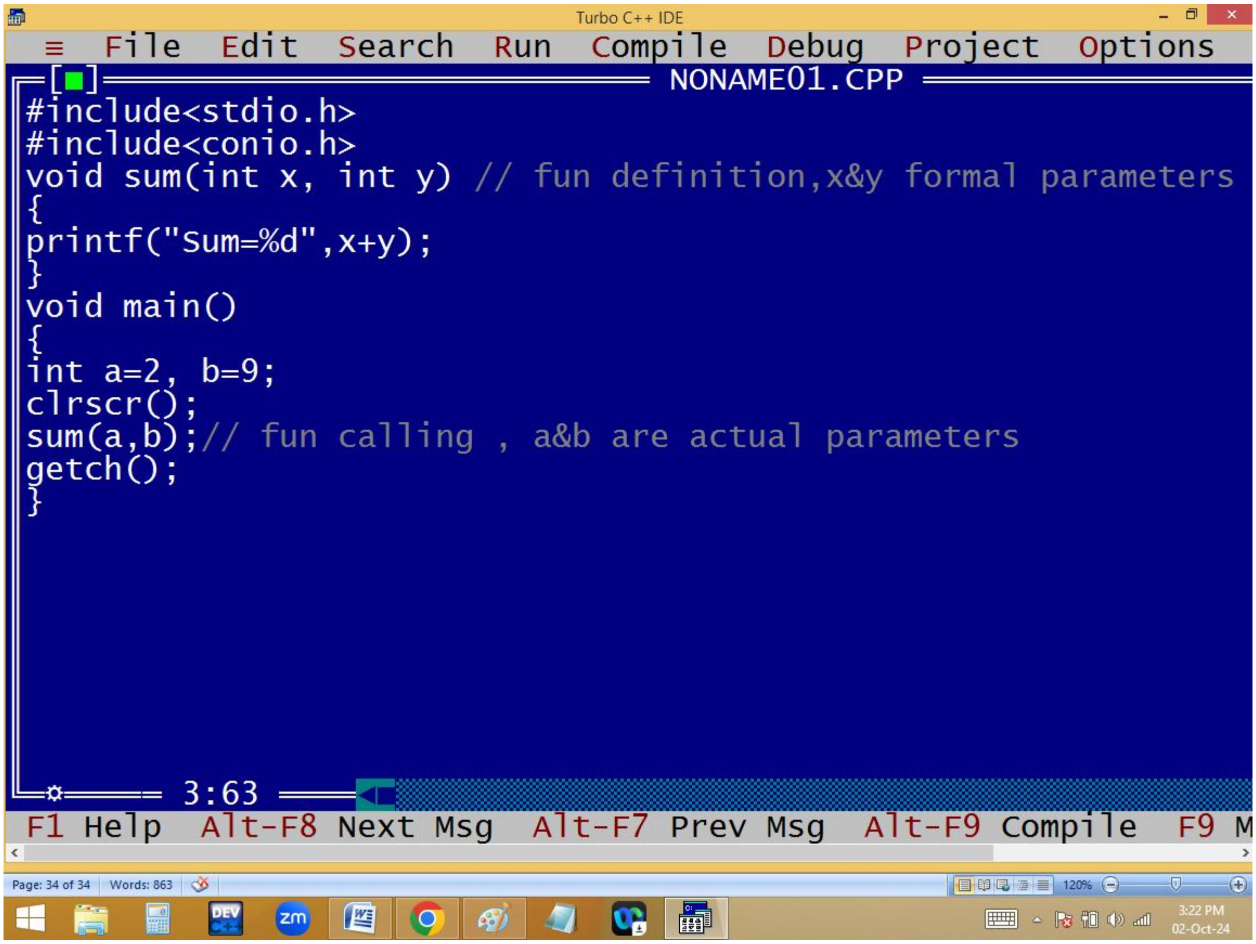

Turbo C++ IDE
Enter a no 4
Even
Even
3:18 PM
02-Oct-24

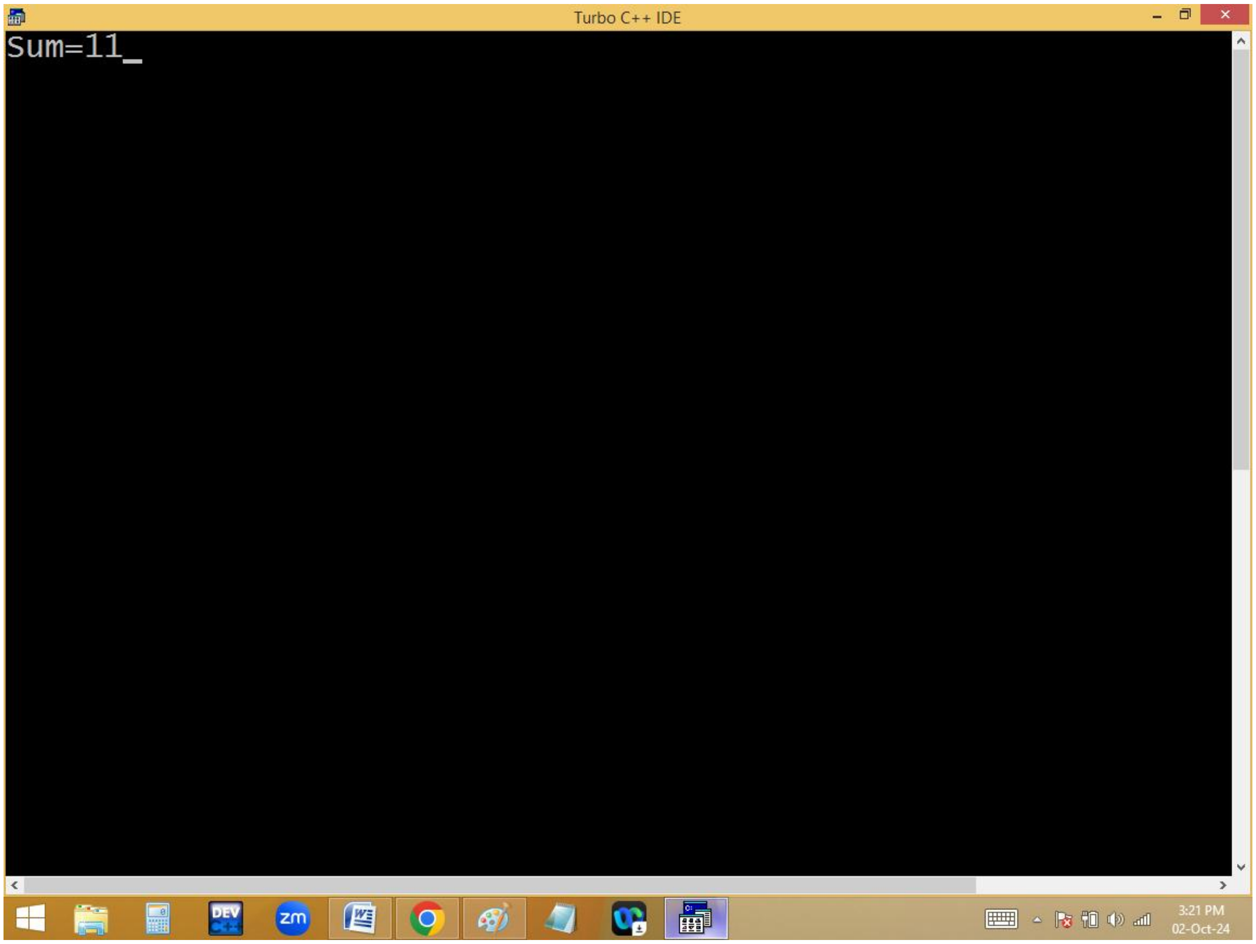

Turbo C++ IDE
Enter a no 5
Odd
Odd
3:18 PM
02-Oct-24

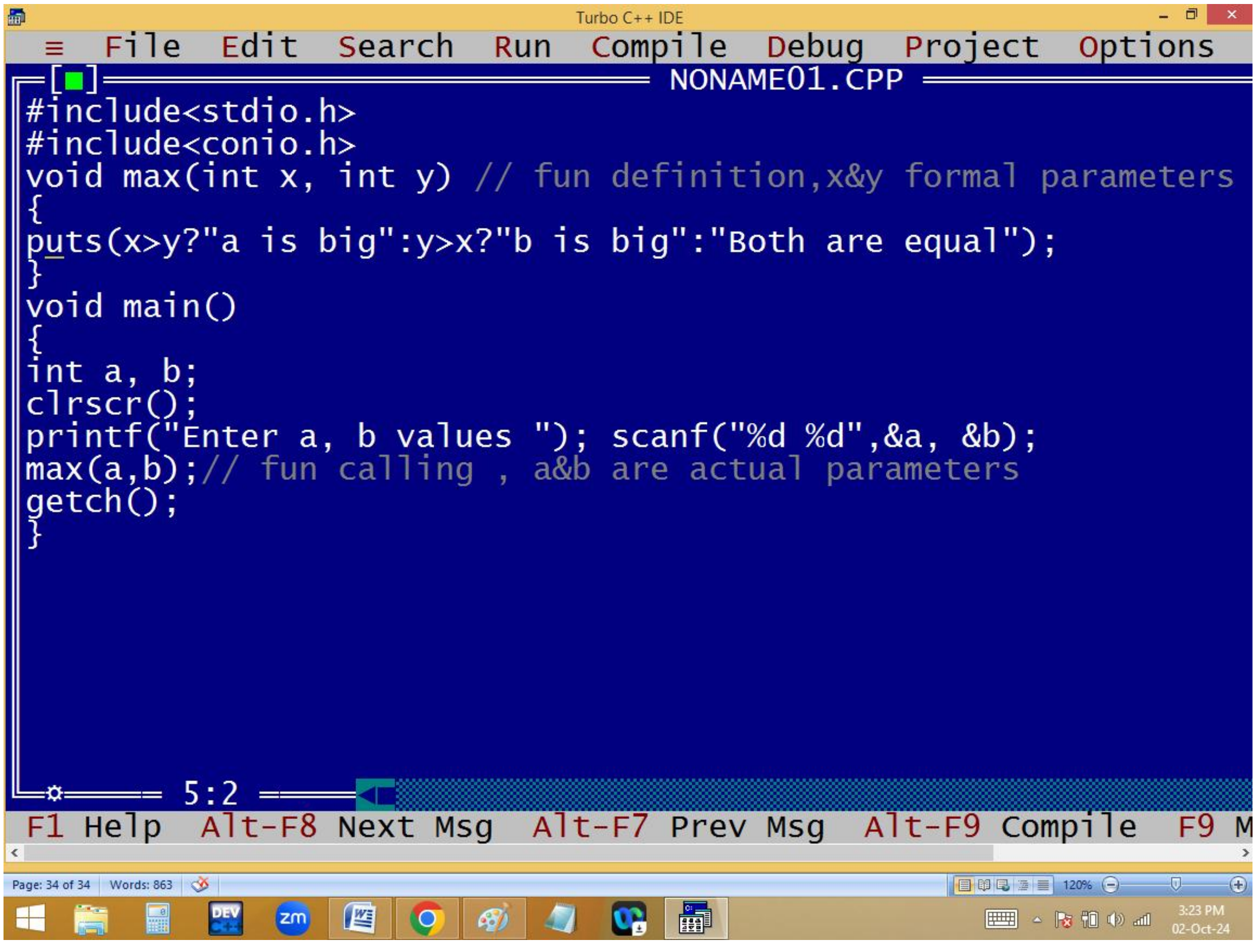

Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
F2.CPP
#include<stdio.h>
#include<conio.h>
int sum(int, int); // arguments or parameters , fun declaratio
int sum(int x, int y) // fun definition, x, y formal parameter
{
return x+y;
}
void main()
{
int a,b;
clrscr();
printf("Enter a, b values "); scanf("%d %d",&a, &b);
int result = sum(a,b); // fun calling, a,b are actual parameter
printf("Sum=%d\n",result);
printf("Sum=%d\n",sum(a,b));
printf("Sum=%d\n",sum(1,2));
printf("Sum=%d\n",sum(11,25));
getch();
}
15:17
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 30 of 30 Words: 858
120%
3:19 PM
02-Oct-24

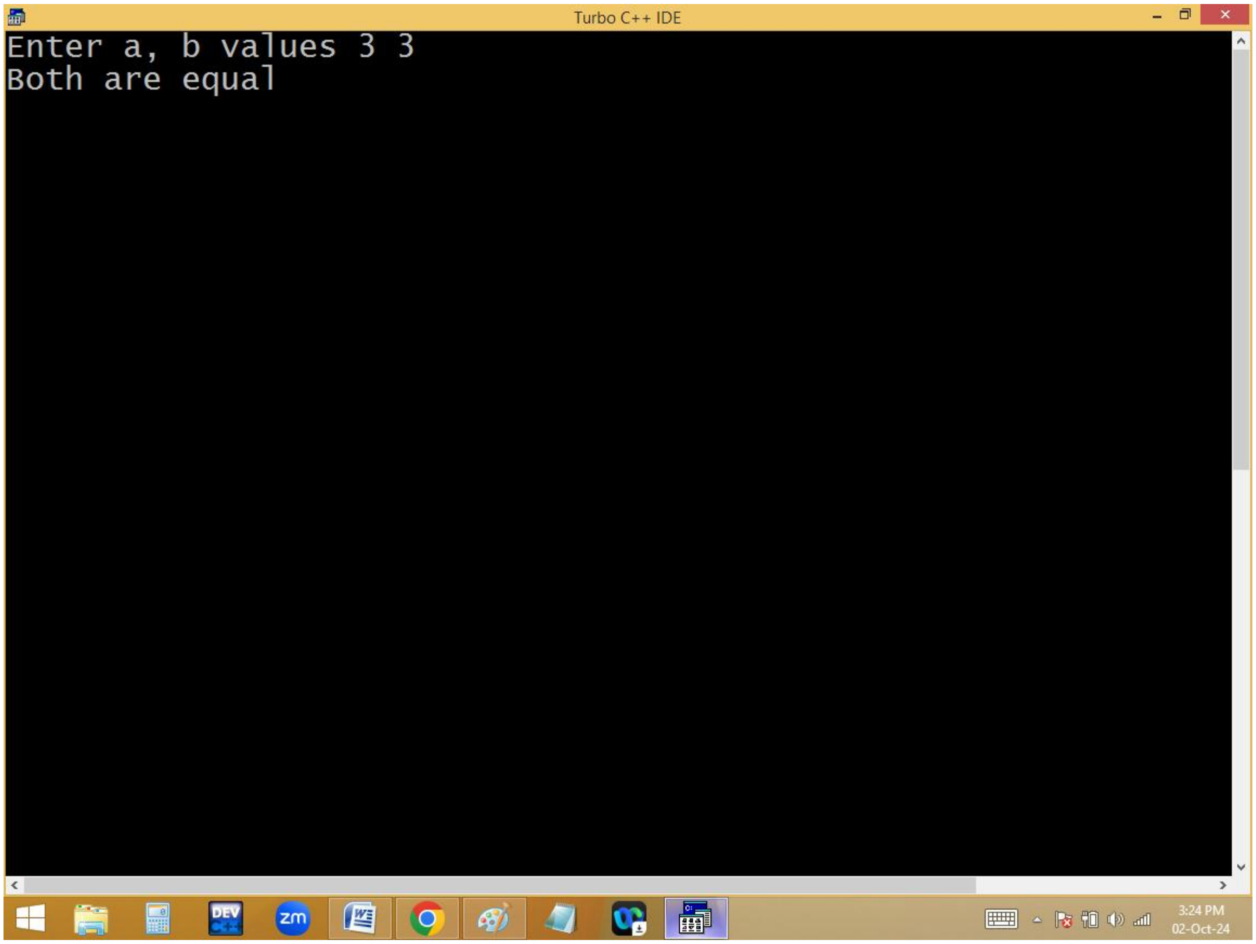


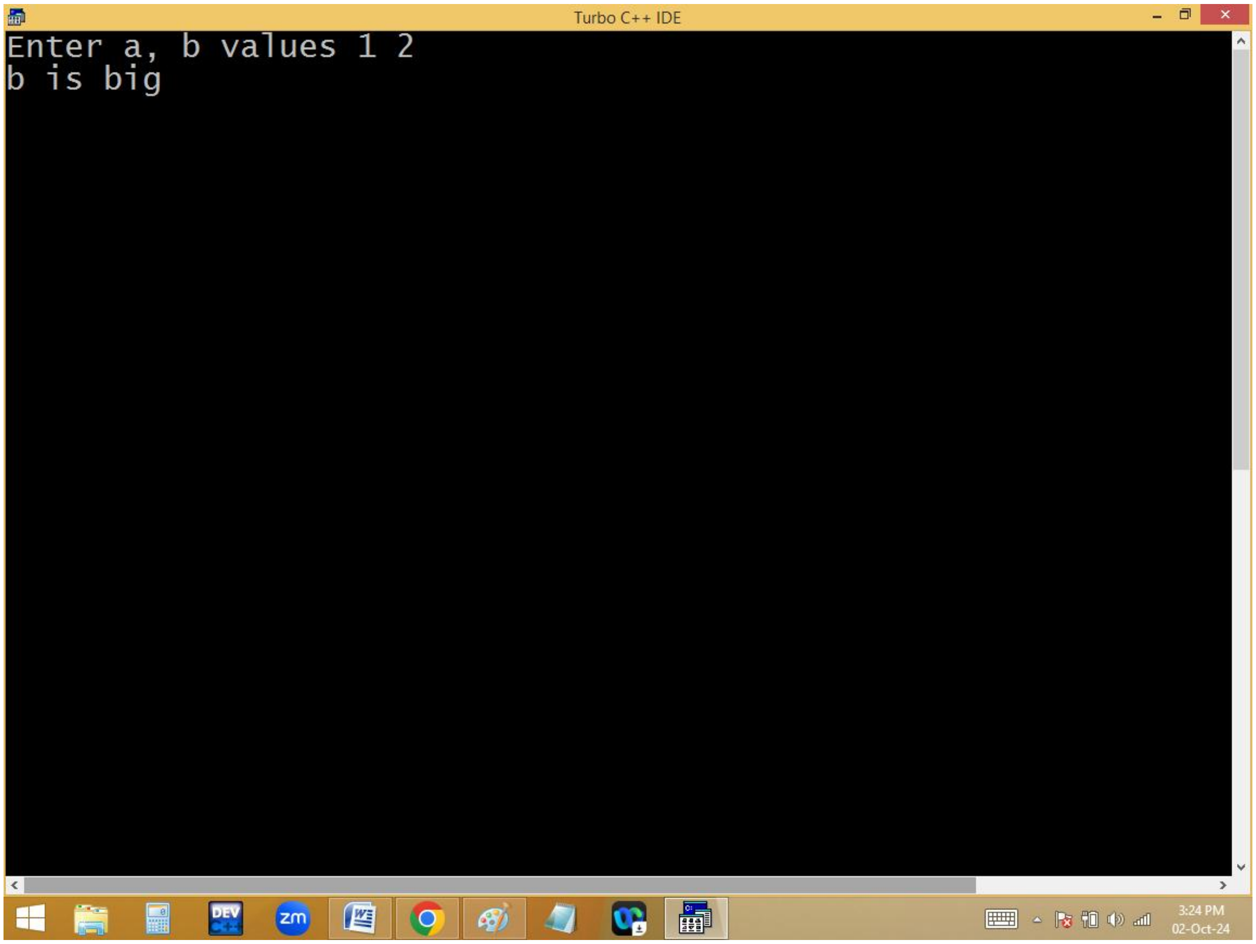
With argument, without return value:

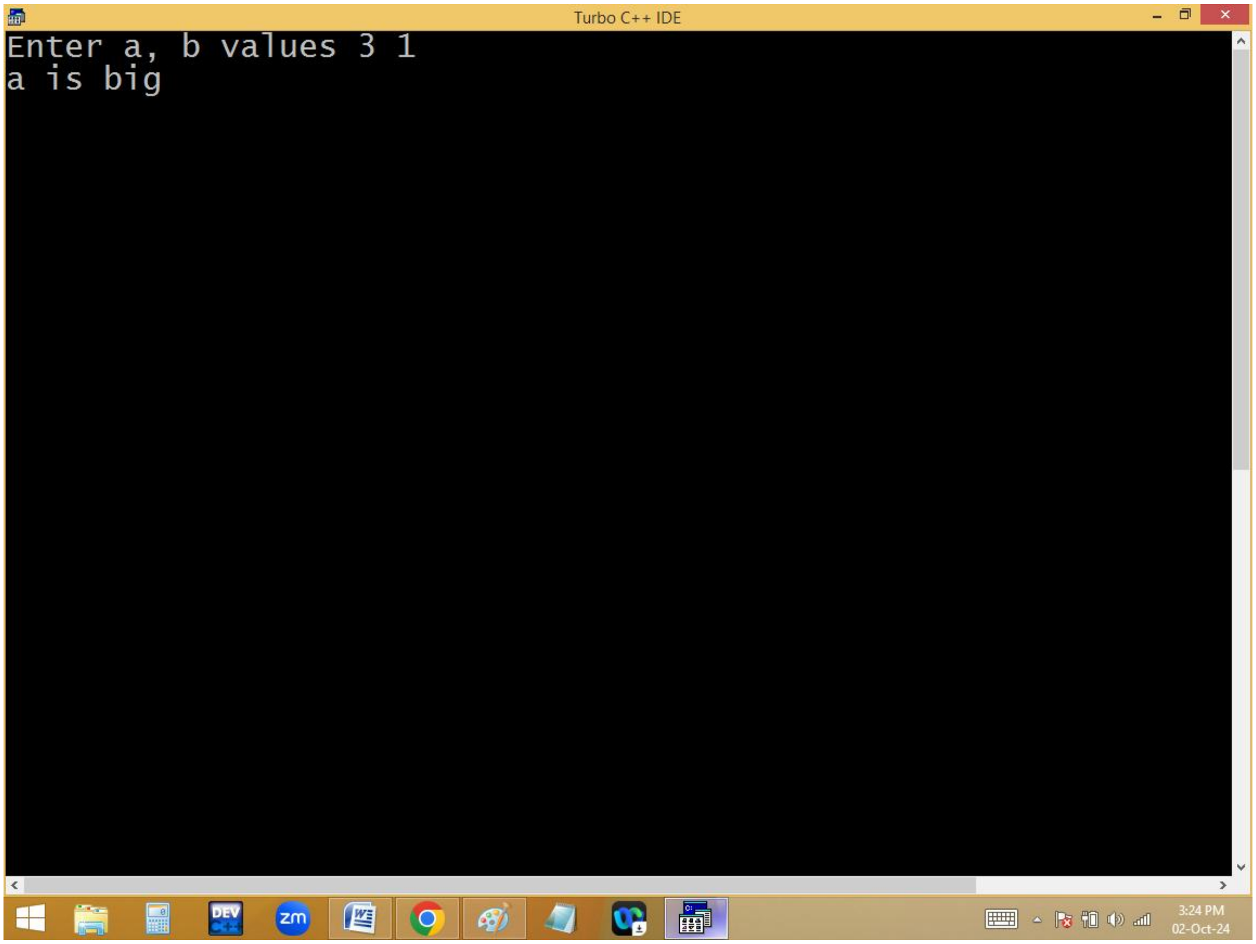

Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
void sum(int x, int y) // fun definition,x&y formal parameters
{
printf("Sum=%d",x+y);
}
void main()
{
int a=2, b=9;
clrscr();
sum(a,b);// fun calling , a&b are actual parameters
getch();
}
3:63
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 34 of 34 Words: 863
120%
3:22 PM
02-Oct-24

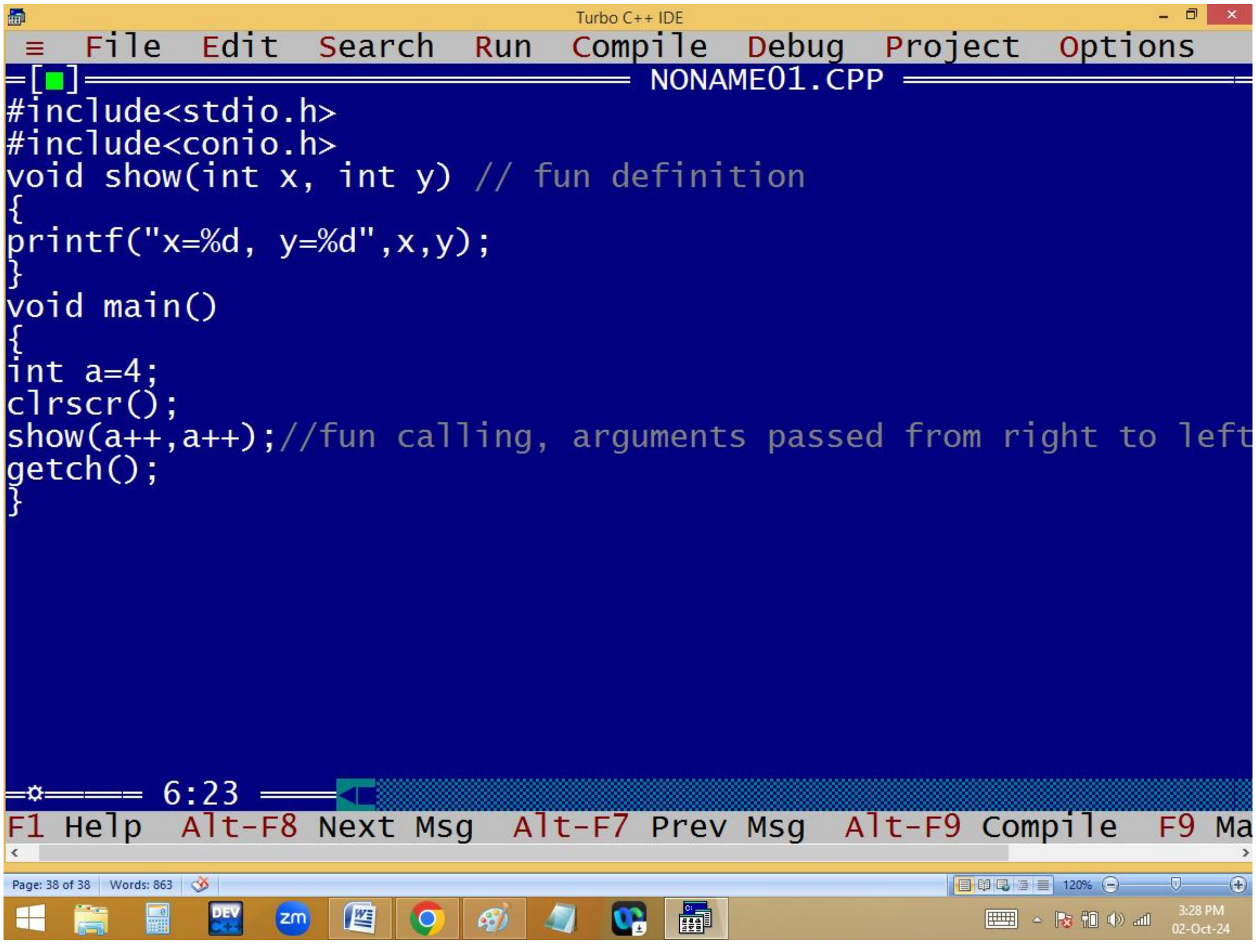

Turbo C++ IDE
Sum=11_
3:21 PM
02-Oct-24


Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
void max(int x, int y) // fun definition,x&y formal parameters
{
puts(x>y?"a is big":y>x?"b is big":"Both are equal");
}
void main()
{
int a, b;
clrscr();
printf("Enter a, b values "); scanf("%d %d",&a, &b);
max(a,b);// fun calling , a&b are actual parameters
getch();
}
5:2
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 34 of 34 Words: 863
120%
3:23 PM
02-Oct-24

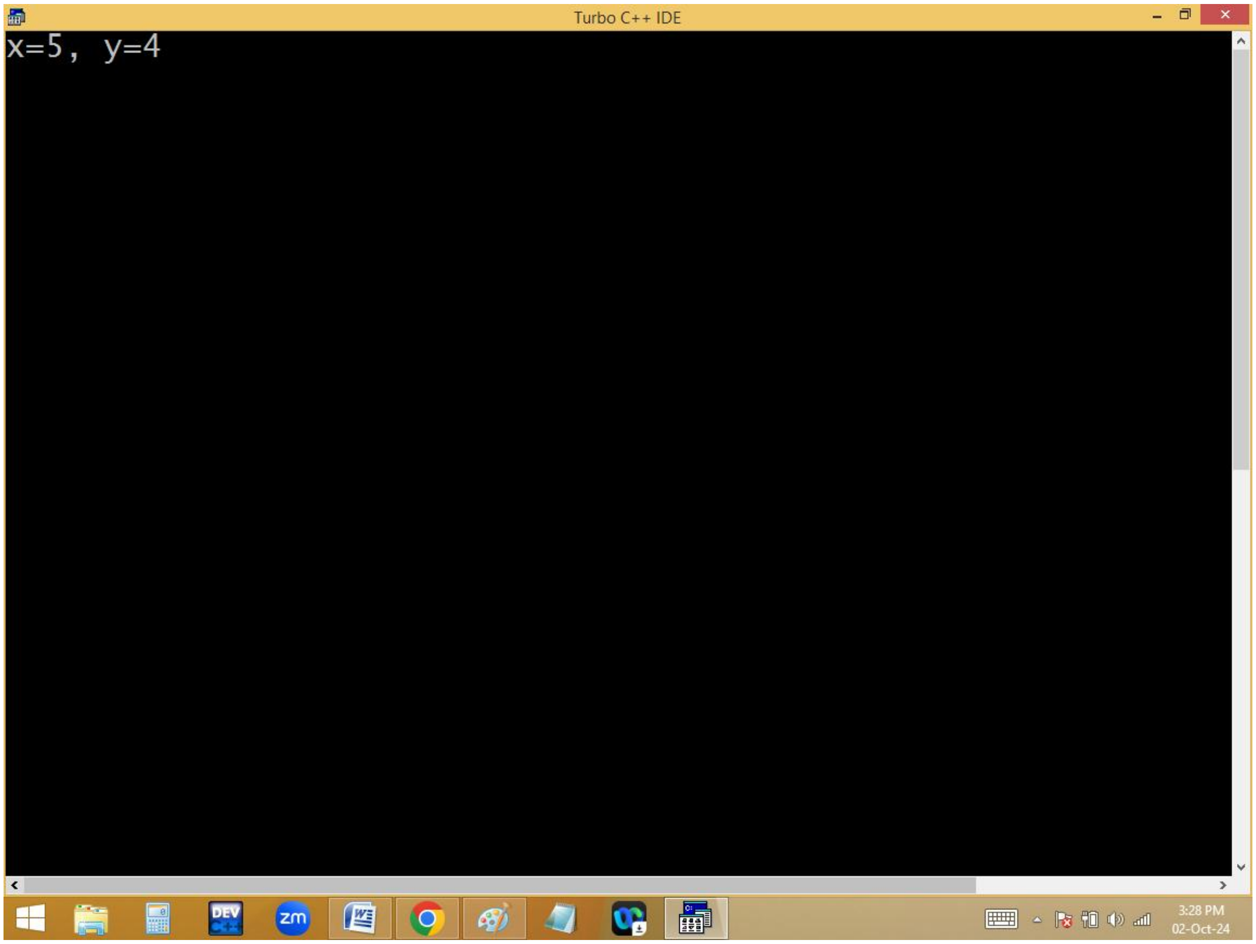

Turbo C++ IDE
Enter a, b values 3 3
Both are equal
3:24 PM
02-Oct-24


Turbo C++ IDE
Enter a, b values 1 2
b is big
3:24 PM
02-Oct-24

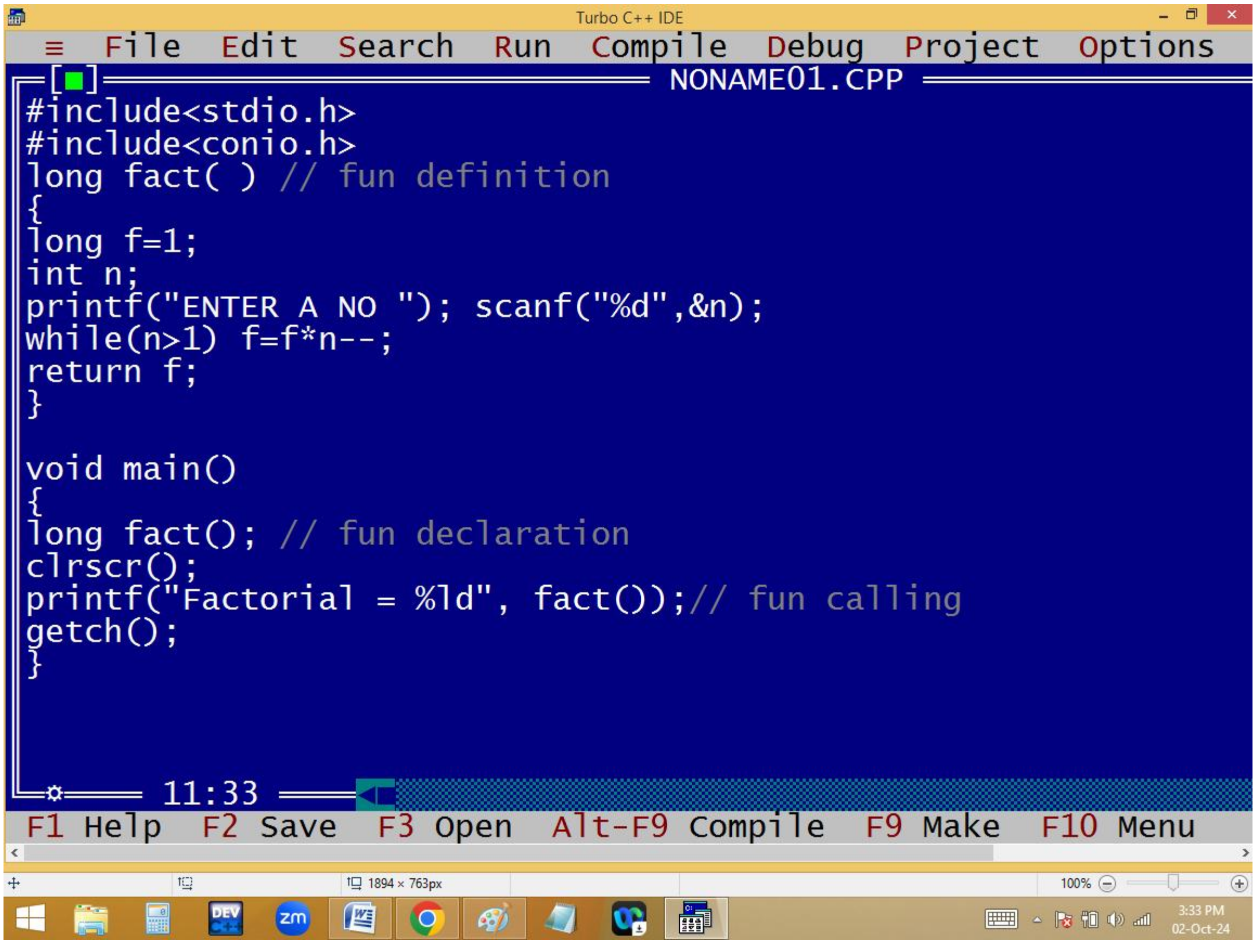

Turbo C++ IDE
Enter a, b values 3 1
a is big
3:24 PM
02-Oct-24



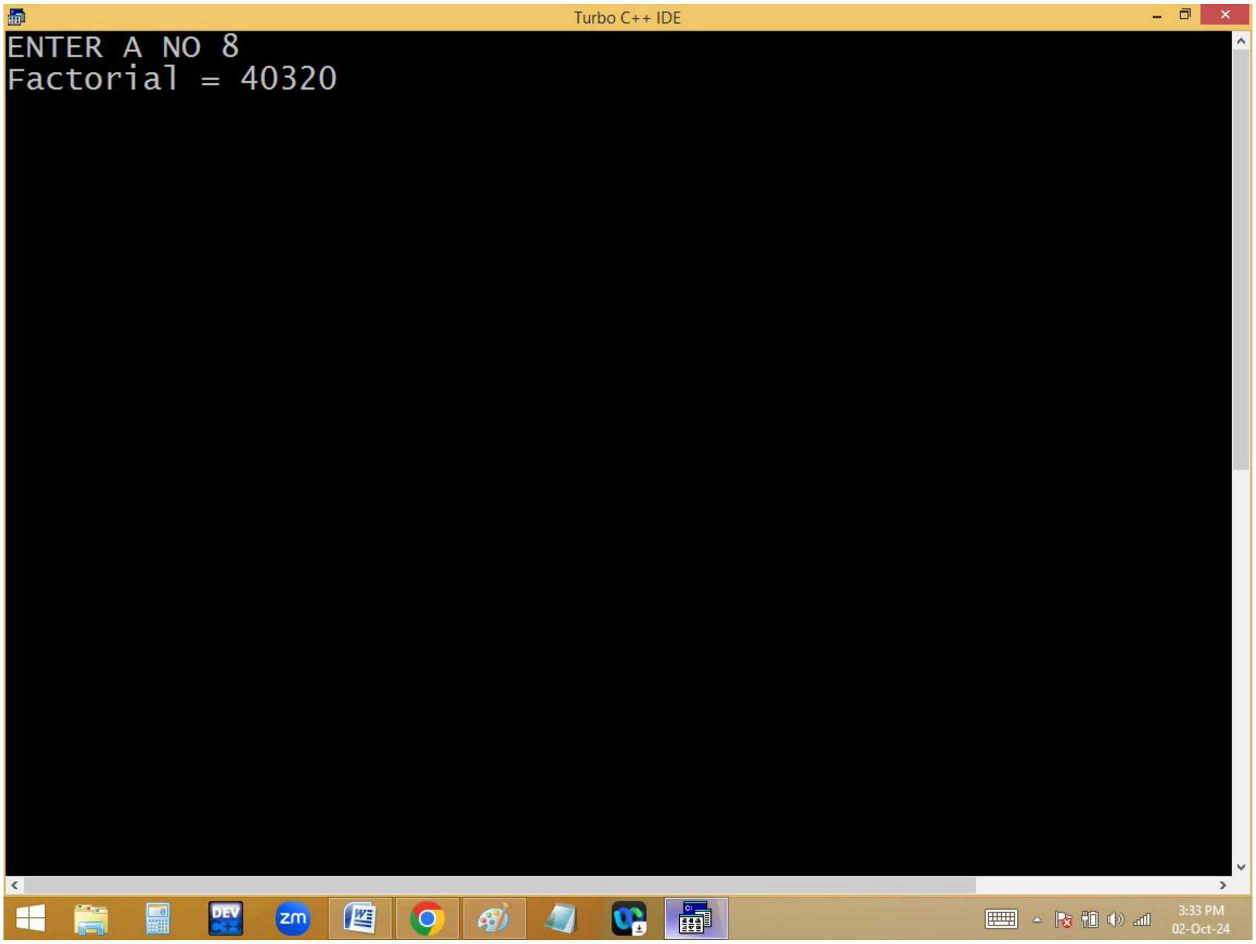

```
#include<stdio.h>
#include<conio.h>
void show(int x, int y) // fun definition
{
printf("x=%d, y=%d",x,y);
}
void main()
{
int a=4;
clrscr();
show(a++,a++);//fun calling, arguments passed from right to left
getch();
}
```

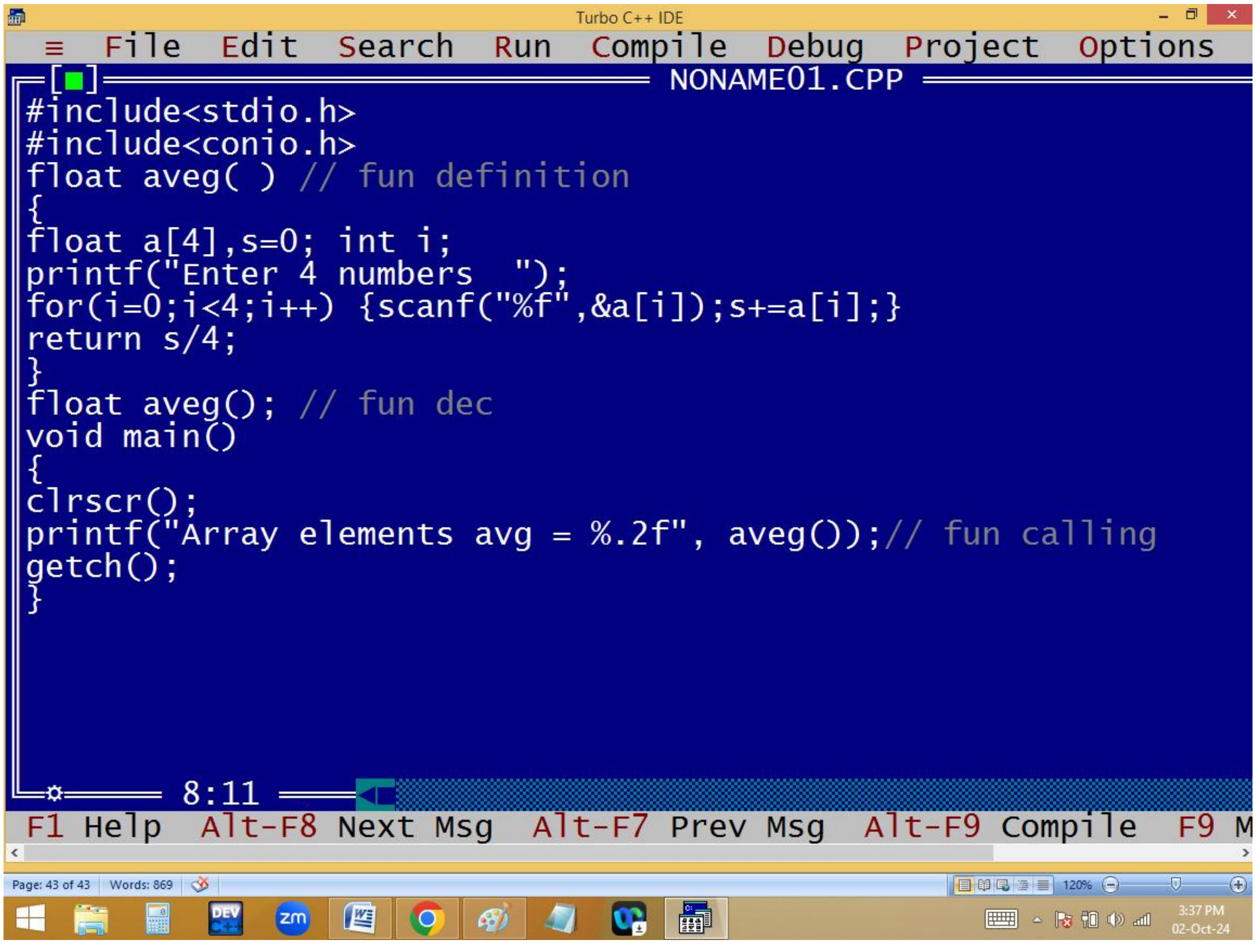


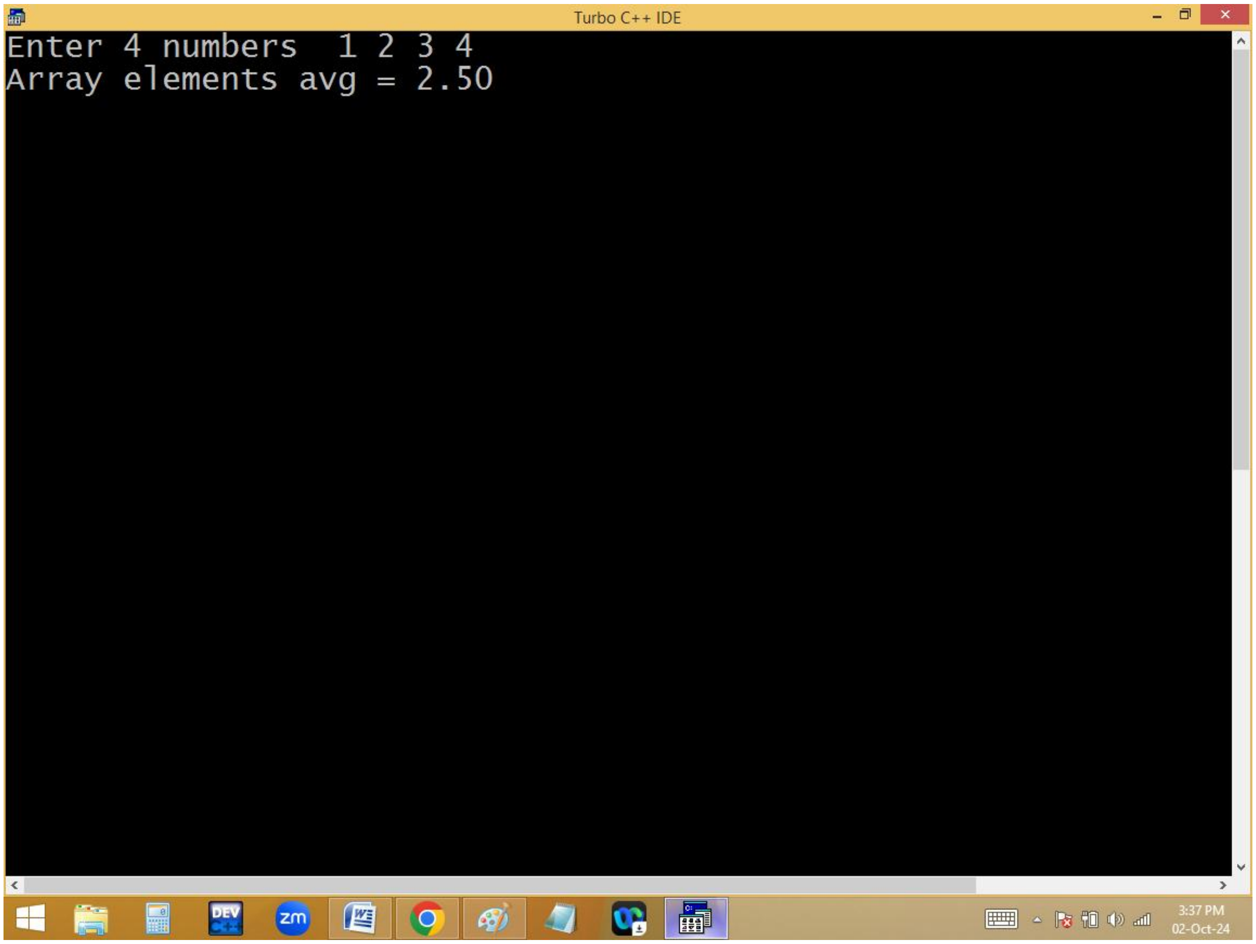
Function without arguments, with return value:

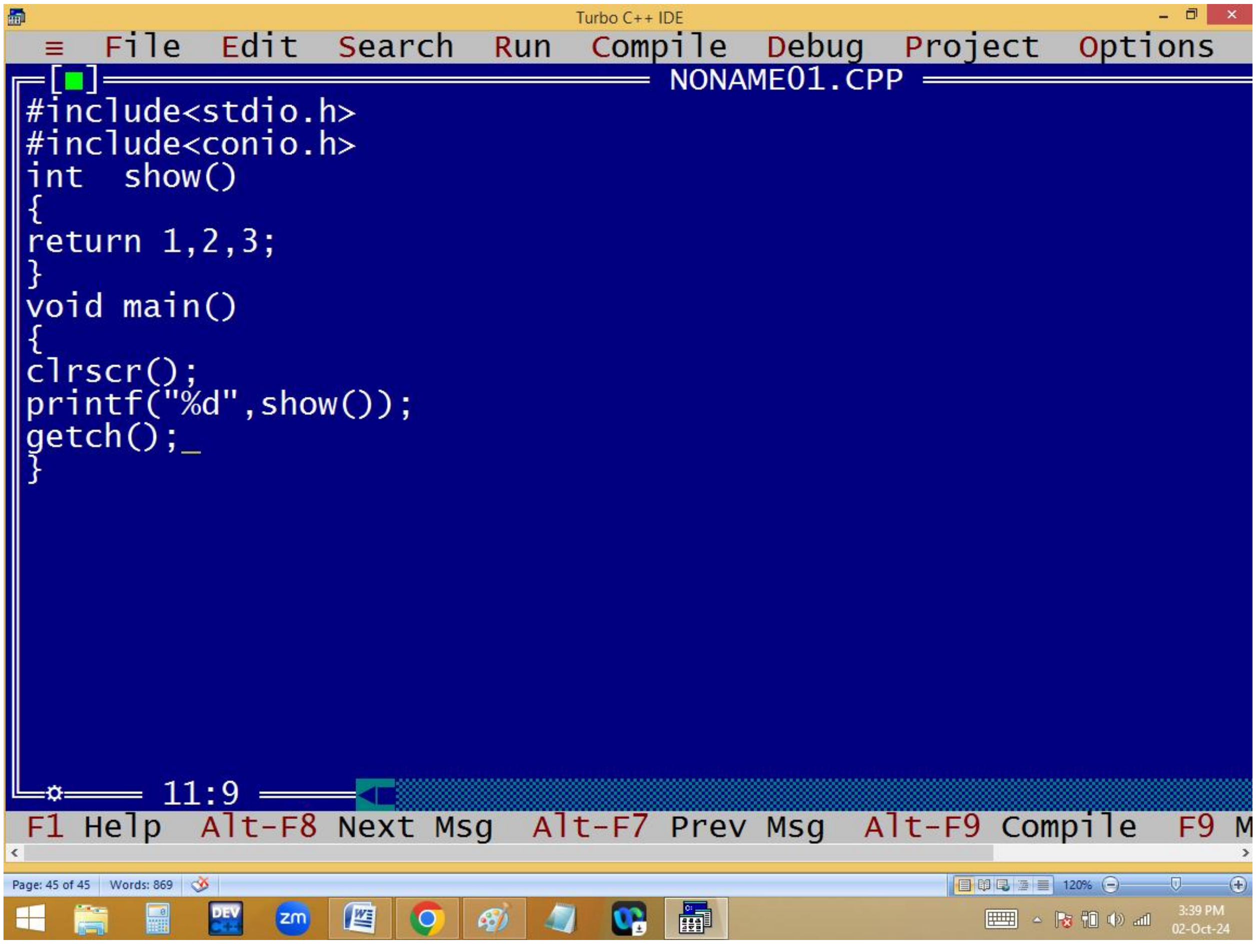

Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
long fact( ) // fun definition
{
long f=1;
int n;
printf("ENTER A NO "); scanf("%d",&n);
while(n>1) f=f*n--;
return f;
}
void main()
{
long fact(); // fun declaration
clrscr();
printf("Factorial = %ld", fact());// fun calling
getch();
}
11:33
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu
1894 × 763px
100%
3:33 PM
02-Oct-24

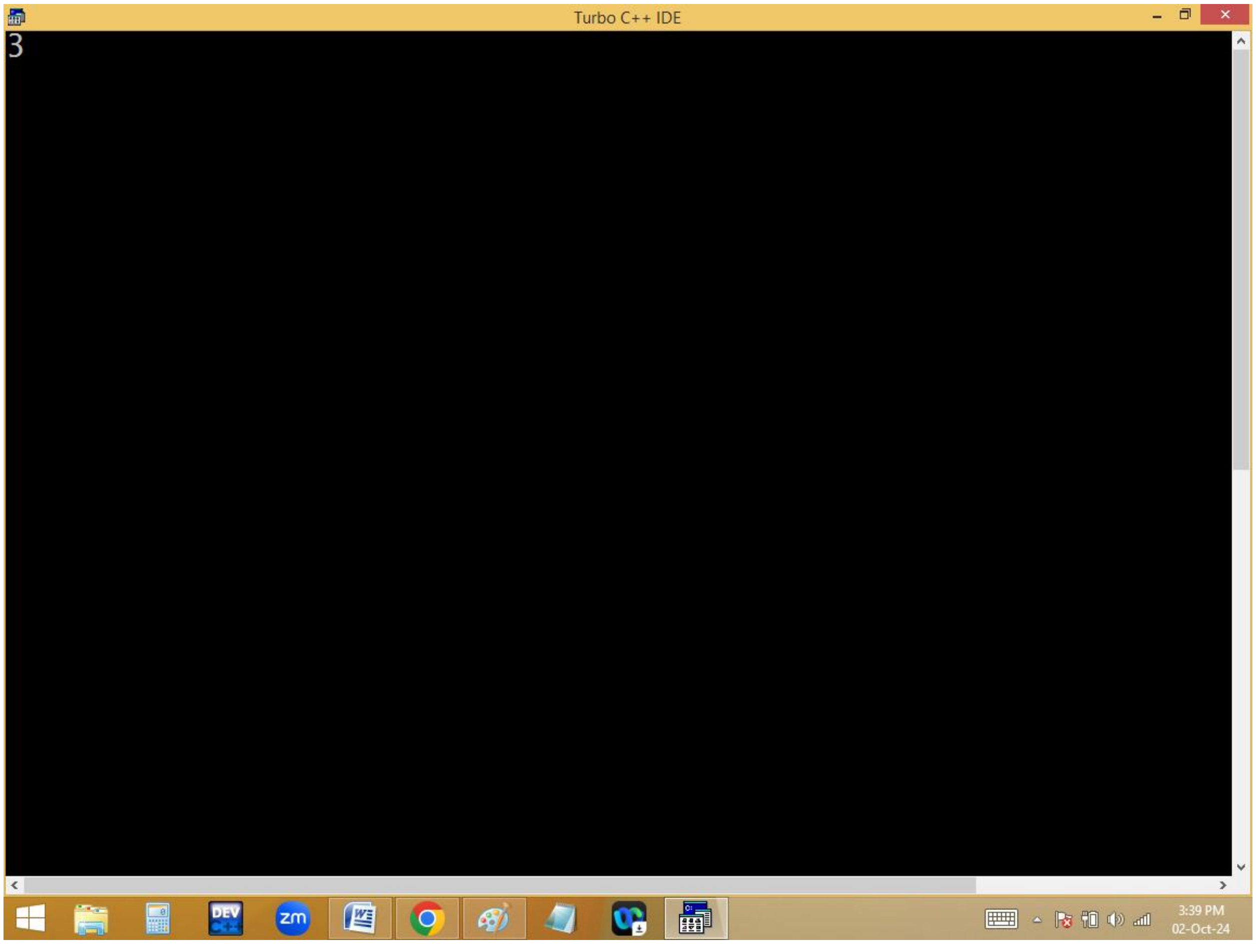

Turbo C++ IDE
ENTER A NO 4
Factorial = 24
3:33 PM
02-Oct-24

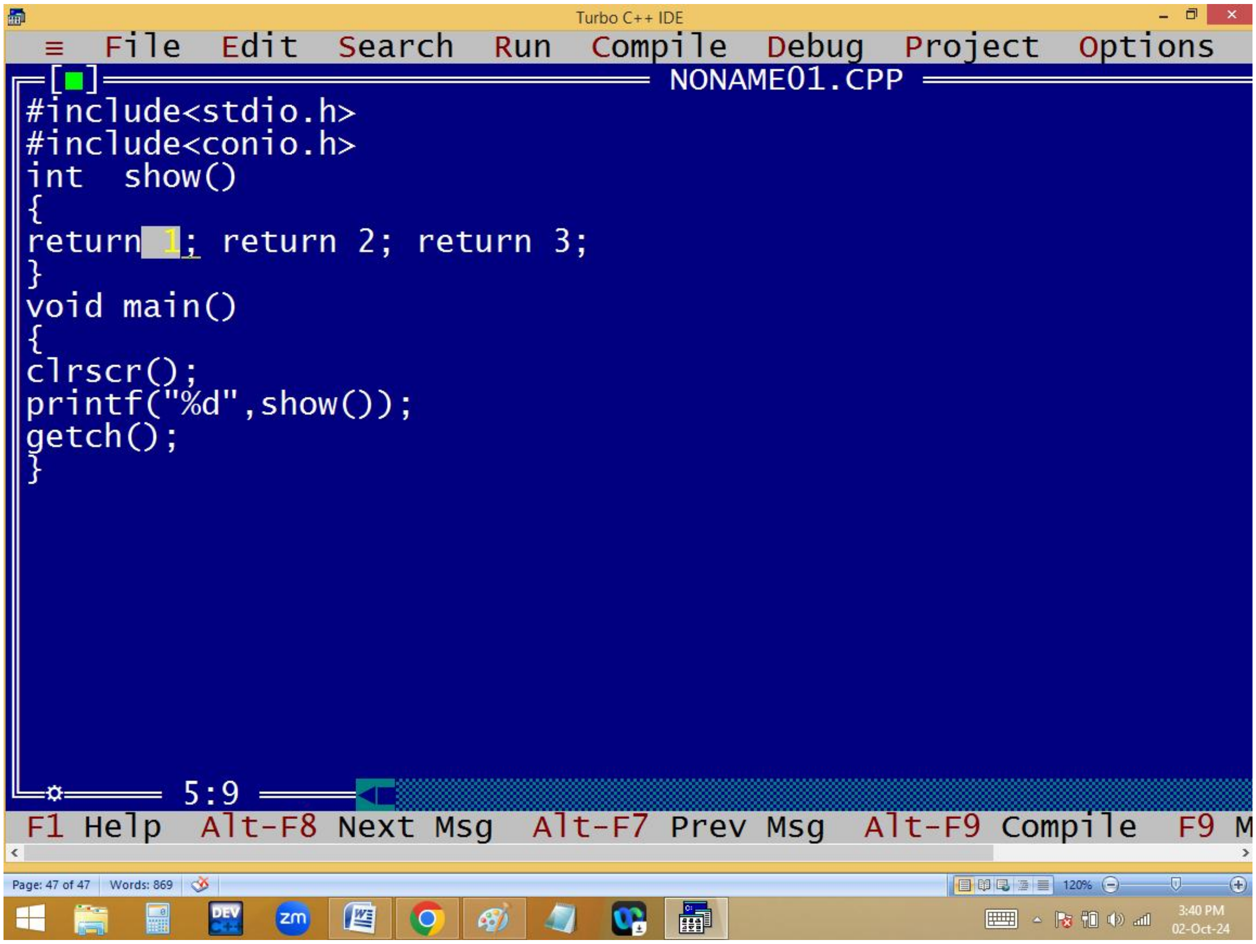

Turbo C++ IDE
ENTER A NO 8
Factorial = 40320
3:33 PM
02-Oct-24

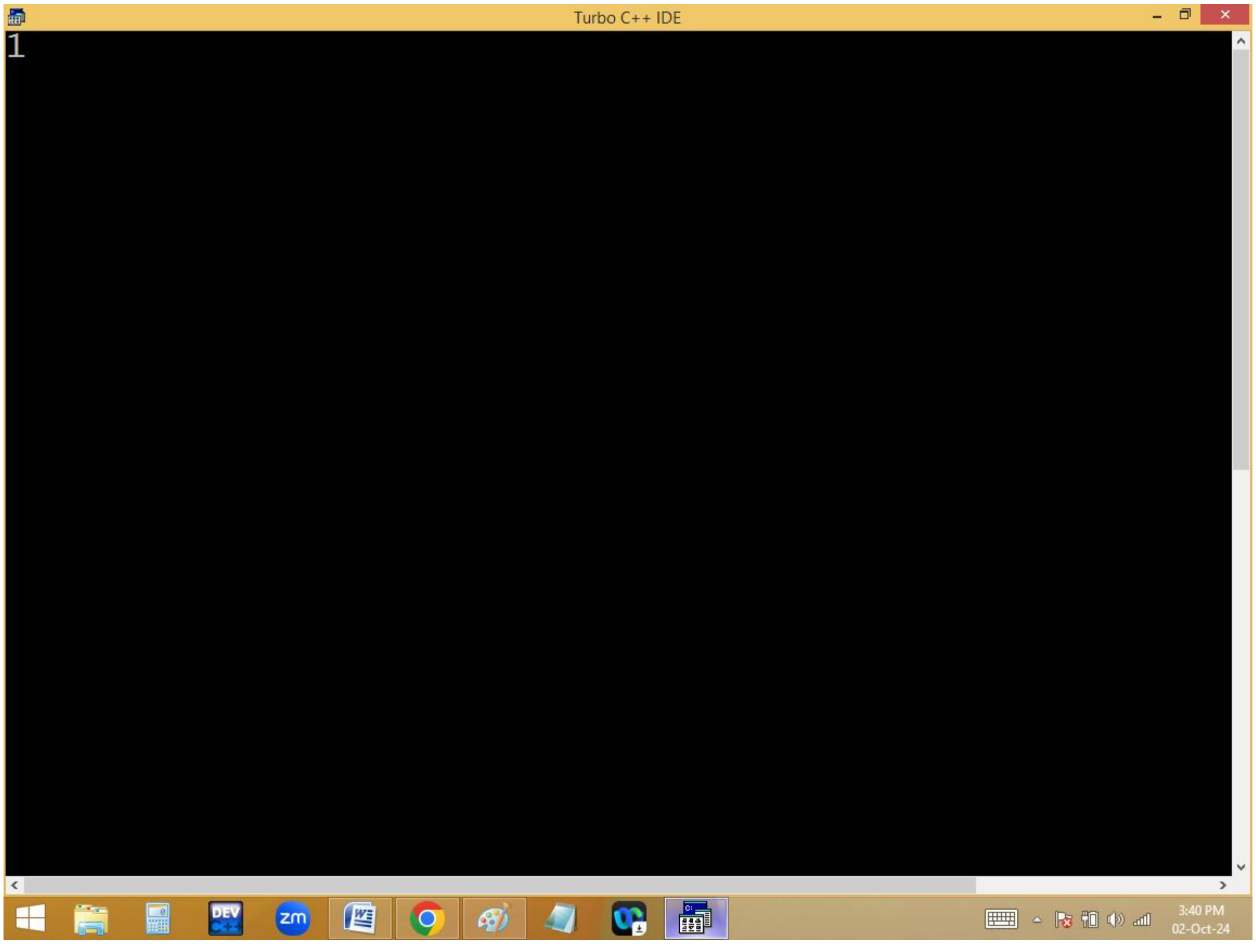

Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
float aveg( ) // fun definition
{
float a[4],s=0; int i;
printf("Enter 4 numbers ");
for(i=0;i<4;i++) {scanf("%f",&a[i]);s+=a[i];}
return s/4;
}
float aveg(); // fun dec
void main()
{
clrscr();
printf("Array elements avg = %.2f", aveg());// fun calling
getch();
}
8:11
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 43 of 43
Words: 869
120%
3:37 PM
02-Oct-24

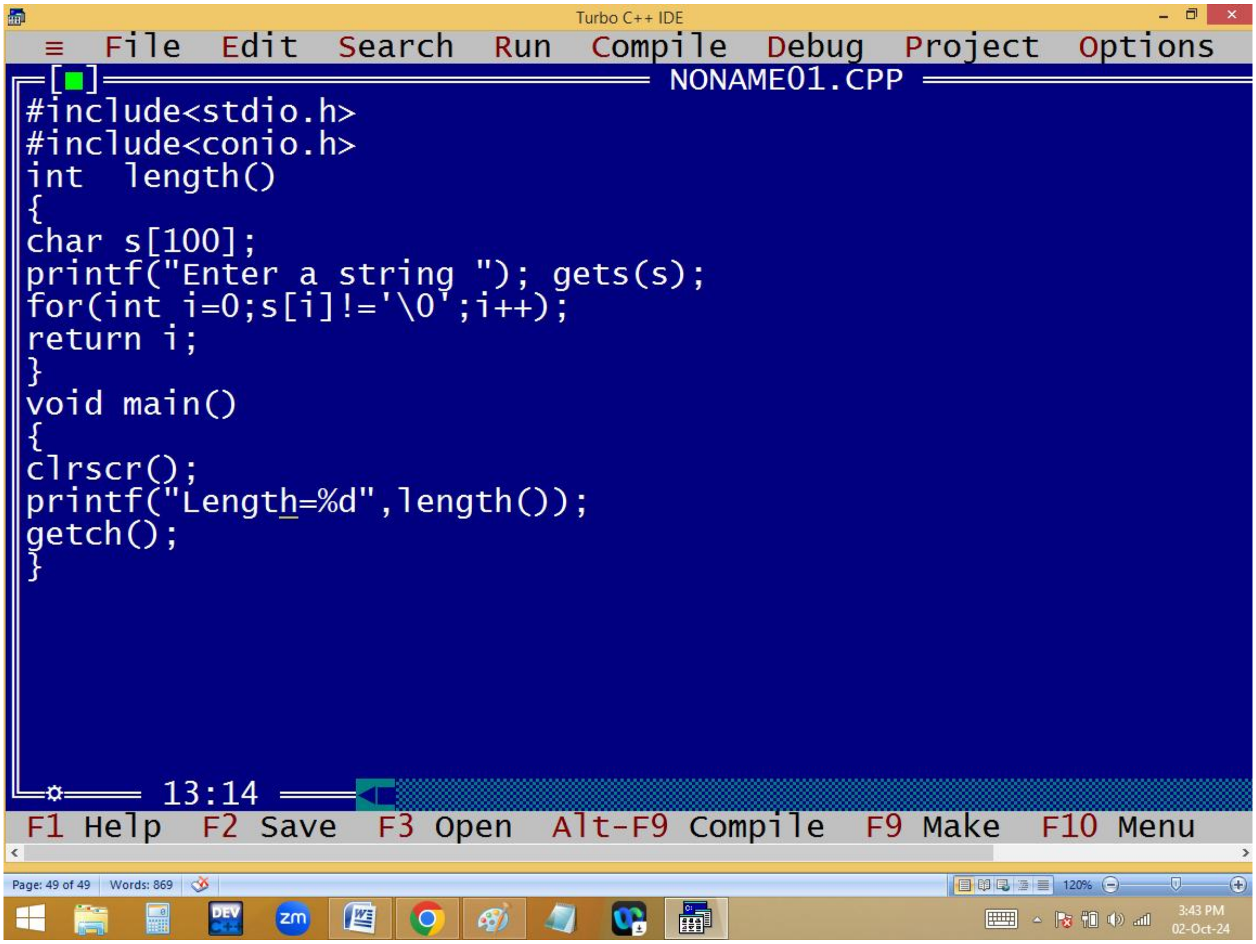

Turbo C++ IDE
Enter 4 numbers 1 2 3 4
Array elements avg = 2.50
3:37 PM
02-Oct-24

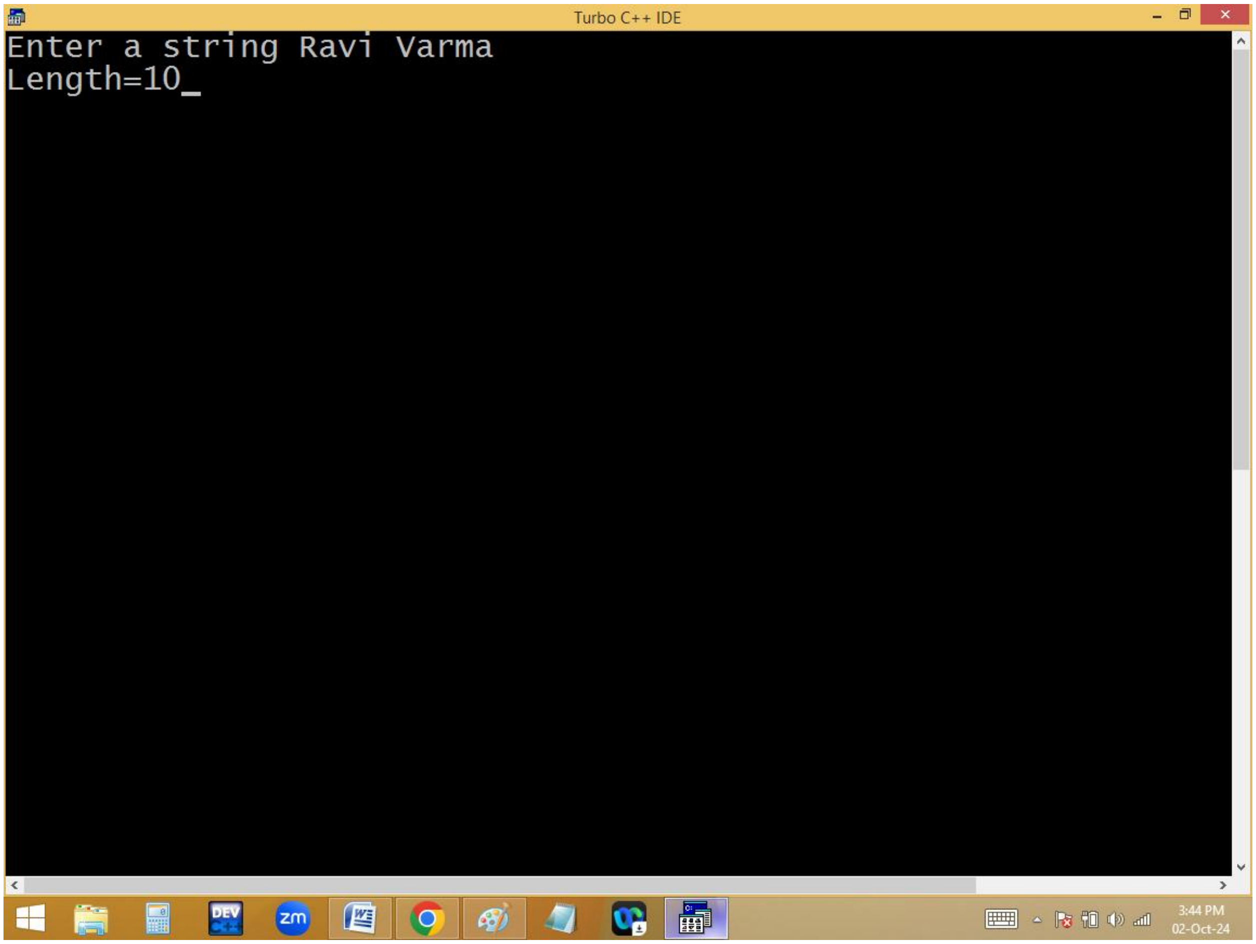

Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
int show()
{
return 1,2,3;
}
void main()
{
clrscr();
printf("%d",show());
getch();_
}
11:9
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 45 of 45
Words: 869
120%
3:39 PM
02-Oct-24


Turbo C++ IDE
3
DEV
zm
3:39 PM
02-Oct-24


Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
[■] NONAME01.CPP
#include<stdio.h>
#include<conio.h>
int show()
{
return 1; return 2; return 3;
}
void main()
{
clrscr();
printf("%d",show());
getch();
}
5:9
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 M
Page: 47 of 47 Words: 869
120%
3:40 PM
02-Oct-24


Turbo C++ IDE
1
3:40 PM
02-Oct-24


Turbo C++ IDE
≡ File Edit Search Run Compile Debug Project Options
NONAME01.CPP
#include<stdio.h>
#include<conio.h>
int length()
{
char s[100];
printf("Enter a string "); gets(s);
for(int i=0;s[i]!='\0';i++);
return i;
}
void main()
{
clrscr();
printf("Length=%d",length());
getch();
}
13:14
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu
Page: 49 of 49 Words: 869
120%
3:43 PM
02-Oct-24


Turbo C++ IDE
Enter a string Ravi Varma
Length=10
3:44 PM
02-Oct-24